

FixItNow

On Demand Home Repair and Maintenance Platform

Full System evaluation Presenatation - Company C

MSD Project | June 2026 |

01 Project Overview & Problem Statement

02 Technology Stack

03 System Architecture

04 Data Model & Database

05 Security & Authentication

06 Core Features & User Flows

07 Frontend (UI/UX, Routing, Responsiveness)

08 Backend (APIs, Services, OOP Design)

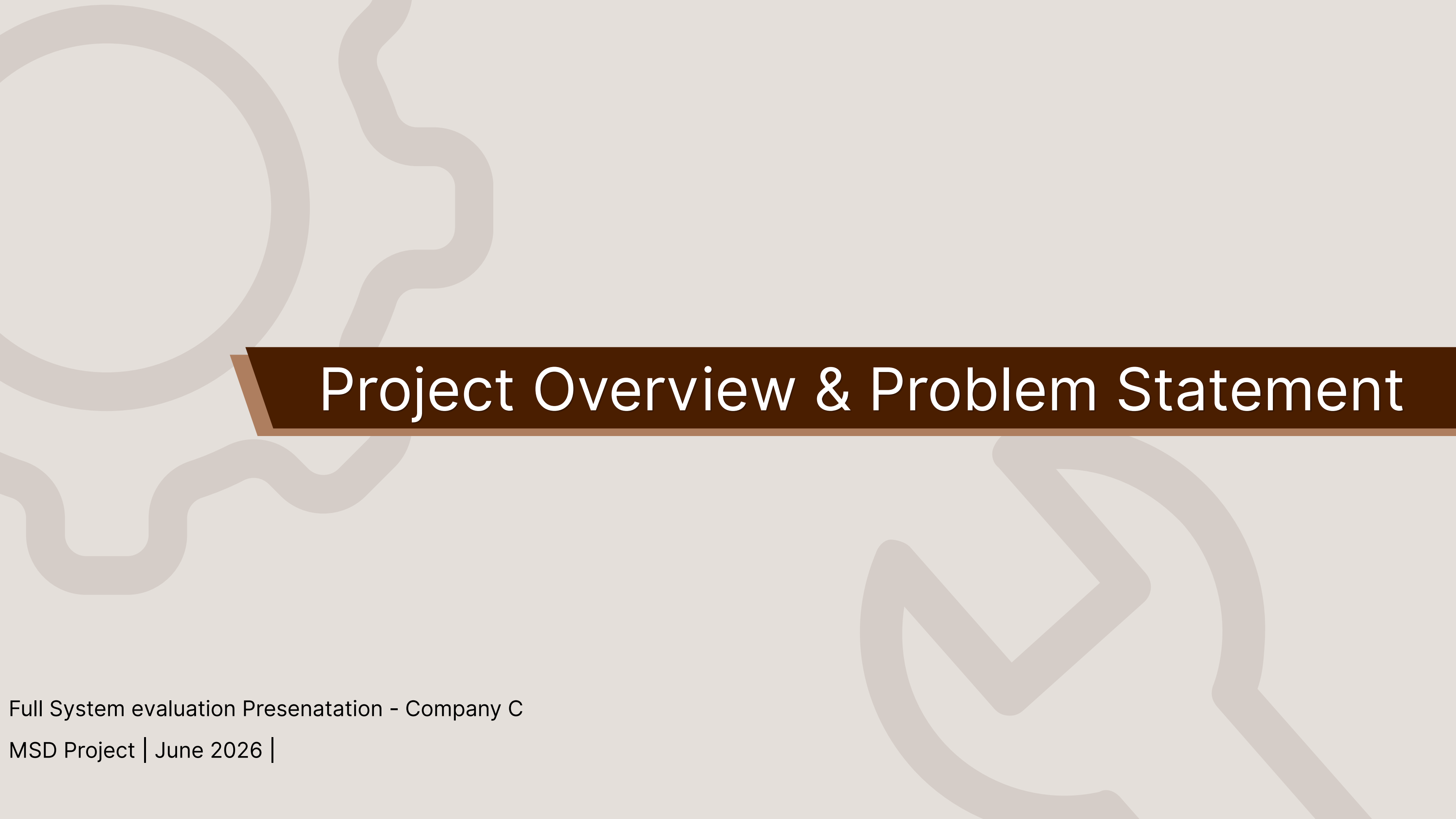
09 Testing & Quality Assurance

10 DevOps — CI/CD Pipeline & Cloud Deployment

11 Performance & Scalability

12 How to Login & Use the System

13 Summary & Q&A



Project Overview & Problem Statement

Full System evaluation Presenatation - Company C
MSD Project | June 2026 |

The Problem we focused

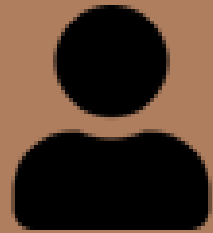
Without FixItNow

- Search unreliable directories
- No way to verify providers
- No real-time tracking
- Manual, slow payment process
- No dispute resolution path

With FixItNow

- Browse verified, rated providers
- Smart geo-ranked matching
- Live GPS job tracking
- Integrated Stripe payments
- Built-in dispute & review system

Actors, Three Coherent Journeys



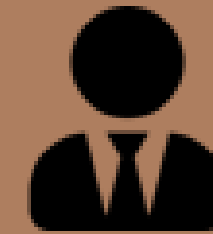
Homeowner

- Register / sign in (email or Google OAuth)
- Browse categories & providers
- Create a booking (on-the-spot or scheduled)
- Track active jobs through its lifecycle
- Confirm completion → leave a review



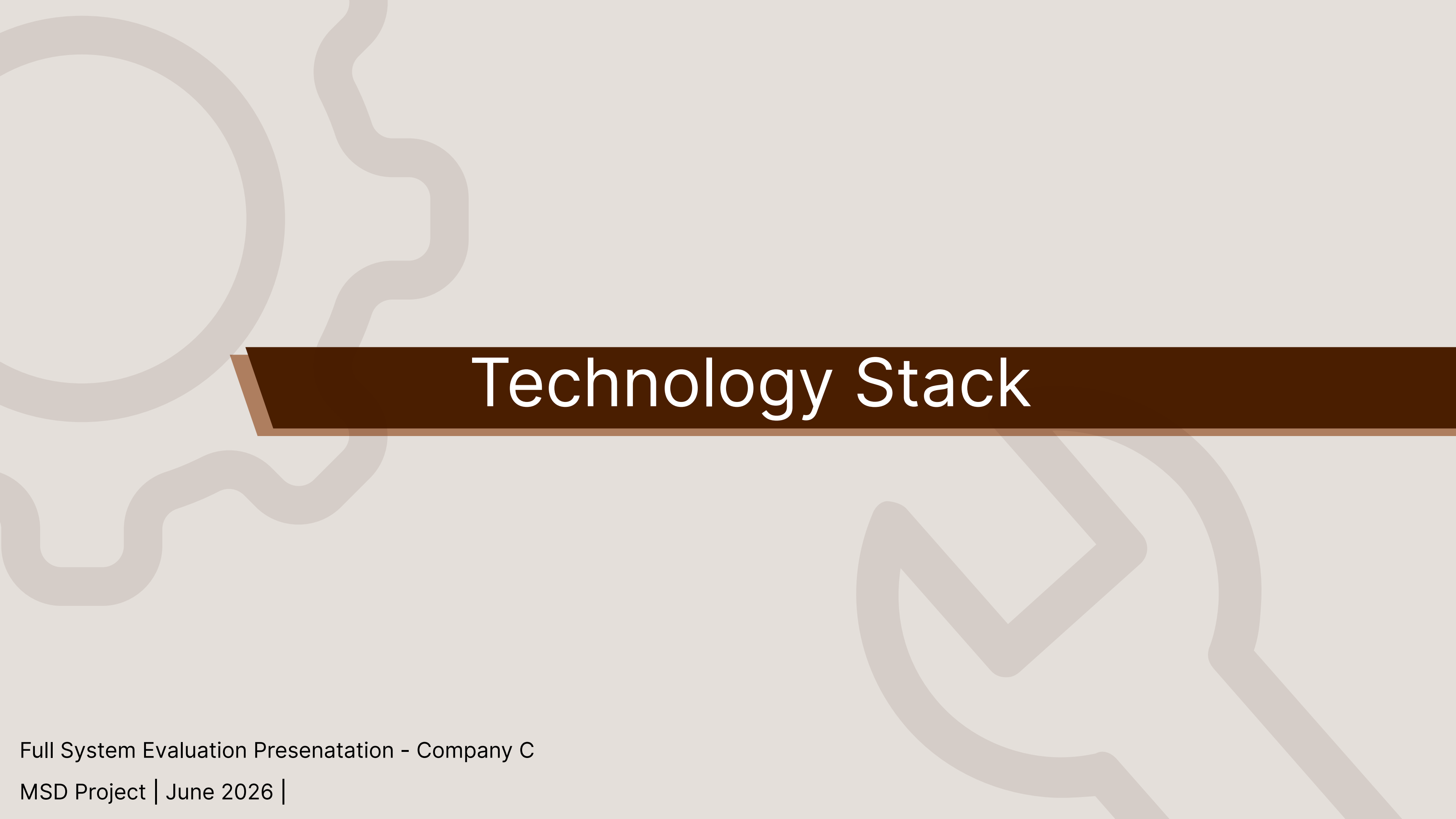
Service Provider

- Apply at sign-up → admin-reviewed ProviderRequest
- Once approved, accept incoming jobs
- View earnings, withdraw from wallet
- Respond to reviews



Administrator

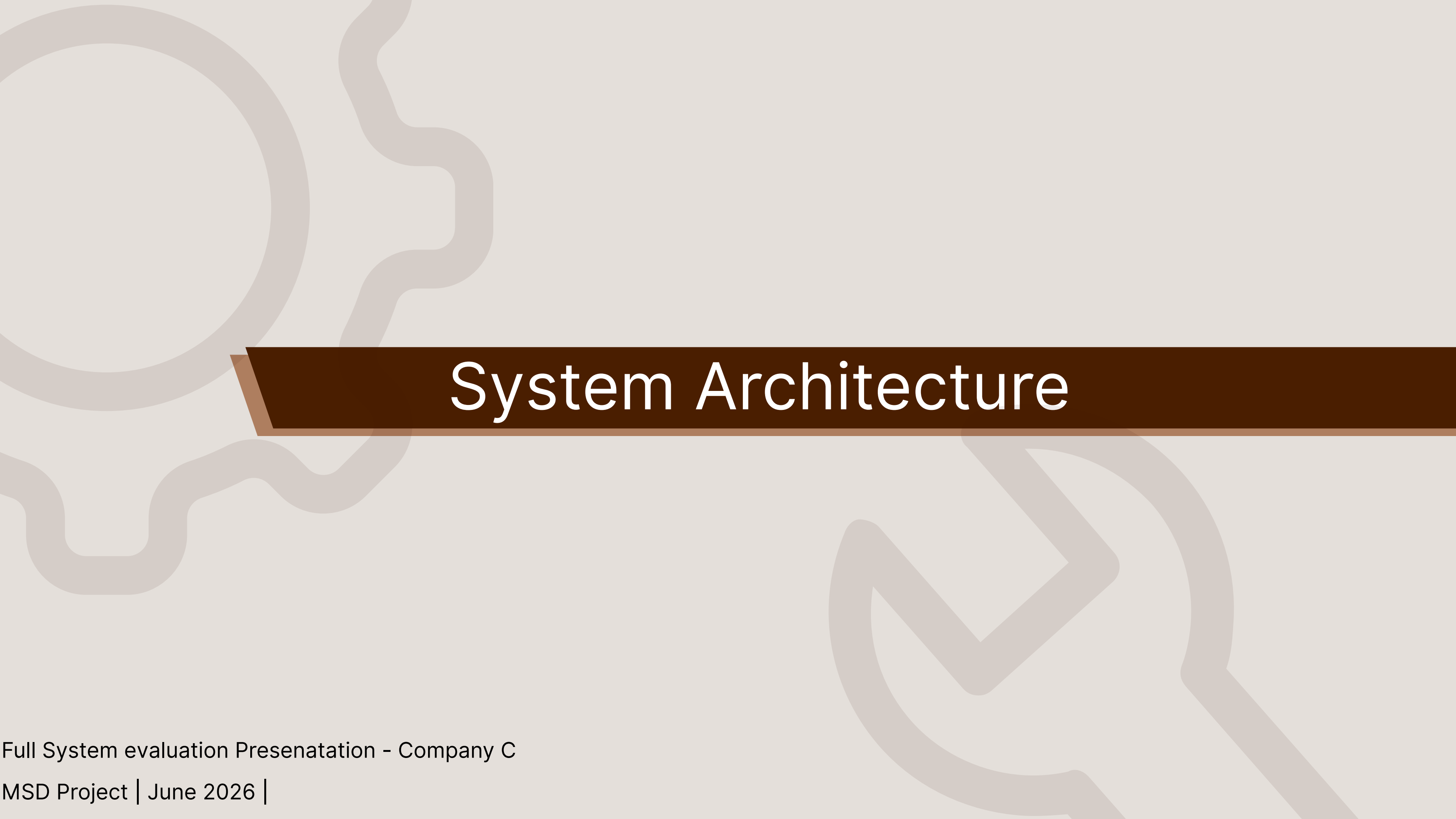
- Review provider & category requests
- Verify, suspend, or reinstate users
- Moderate reviews
- Monitor dashboard, security, system health
- Manage news content



Technology Stack

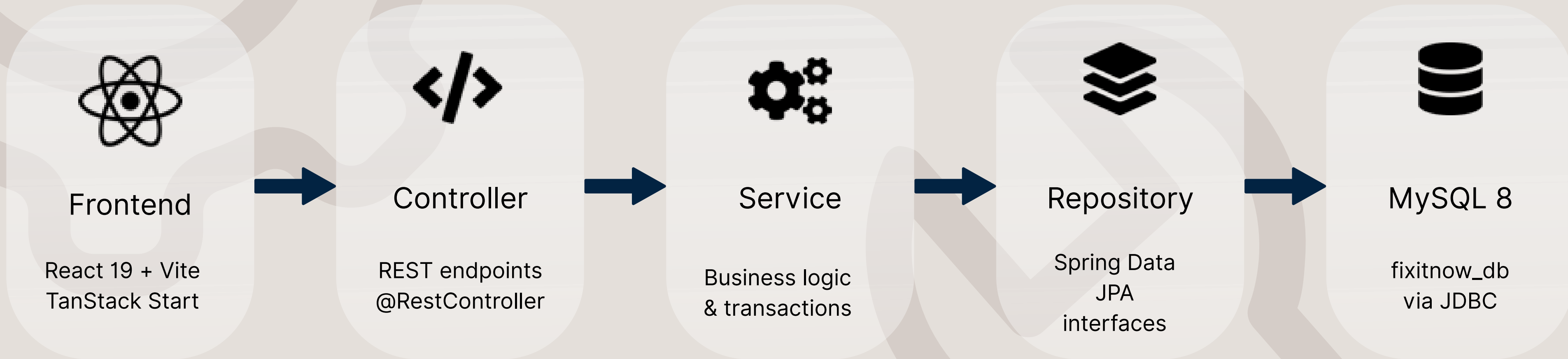
Full Technology Stack

Layer	Technology	Why
Backend Framework	Spring Boot 4 / Java 21	REST API, dependency injection, layered architecture
Database	MySQL 8	Relational persistence via Spring Data JPA
Frontend Framework	React 19 + Vite	Component-driven SPA with fast HMR builds
Routing / Meta-framework	TanStack Start (TanStack Router)	Type safe routing for the TypeScript frontend
Language (Frontend)	TypeScript	Static typing across routes, services, and hooks
Authentication	JWT (access + refresh) + Google OAuth2	Stateless auth; OAuth2AuthorizationRequest via cookie
Real-Time	WebSocket / STOMP (/ws/tracker)	Live GPS broadcast via TrackingService
Testing	JUnit 5, Mockito, AssertJ, MockMvc	Unit, integration, RBAC, and acceptance coverage



System Architecture

A Classic Layered Client-Server Architecture



Package by Layer Organisation



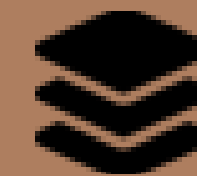
controller/

HTTP → service calls, wrapped in ResponseEntity. BookingV1Controller, ProvidersAdminController, AuthV1Controller.



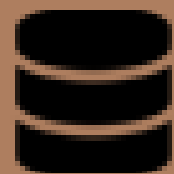
service/

Business rules & transactions. BookingService (state machine), SmartMatchService (ranking), WalletV1Service (ledger).



repository/

14 interfaces extending JpaRepository<T,ID> UserRepository, BookingRepository, RefreshTokenRepository.



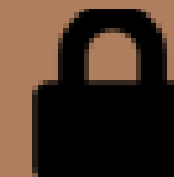
entity/

JPA @Entity domain objects: User, Booking, Service, Category, Payment, Review, Notification, Dispute, WalletTransaction.



dto/

API contract objects - request, response (ApiResponse<T>), and the v1 frontend-facing contract



security/

JwtAuthFilter, JwtTokenProvider, CustomUserDetailsService, UserPrincipal, OAuth2LoginSuccessHandler.

Ex: Booking Request lifecycle

1

Frontend call

createBooking() in lib/booking.ts → api-client.ts attaches Authorization: Bearer <access_token>

2

Security filter

JwtAuthFilter (extends OncePerRequestFilter) validates the token via JwtTokenProvider and populates the SecurityContext with a UserPrincipal

3

Controller

BookingV1Controller deserialises JSON into CreateBookingRequestV1 (@Valid) and resolves the current user id via SecurityUtil

4

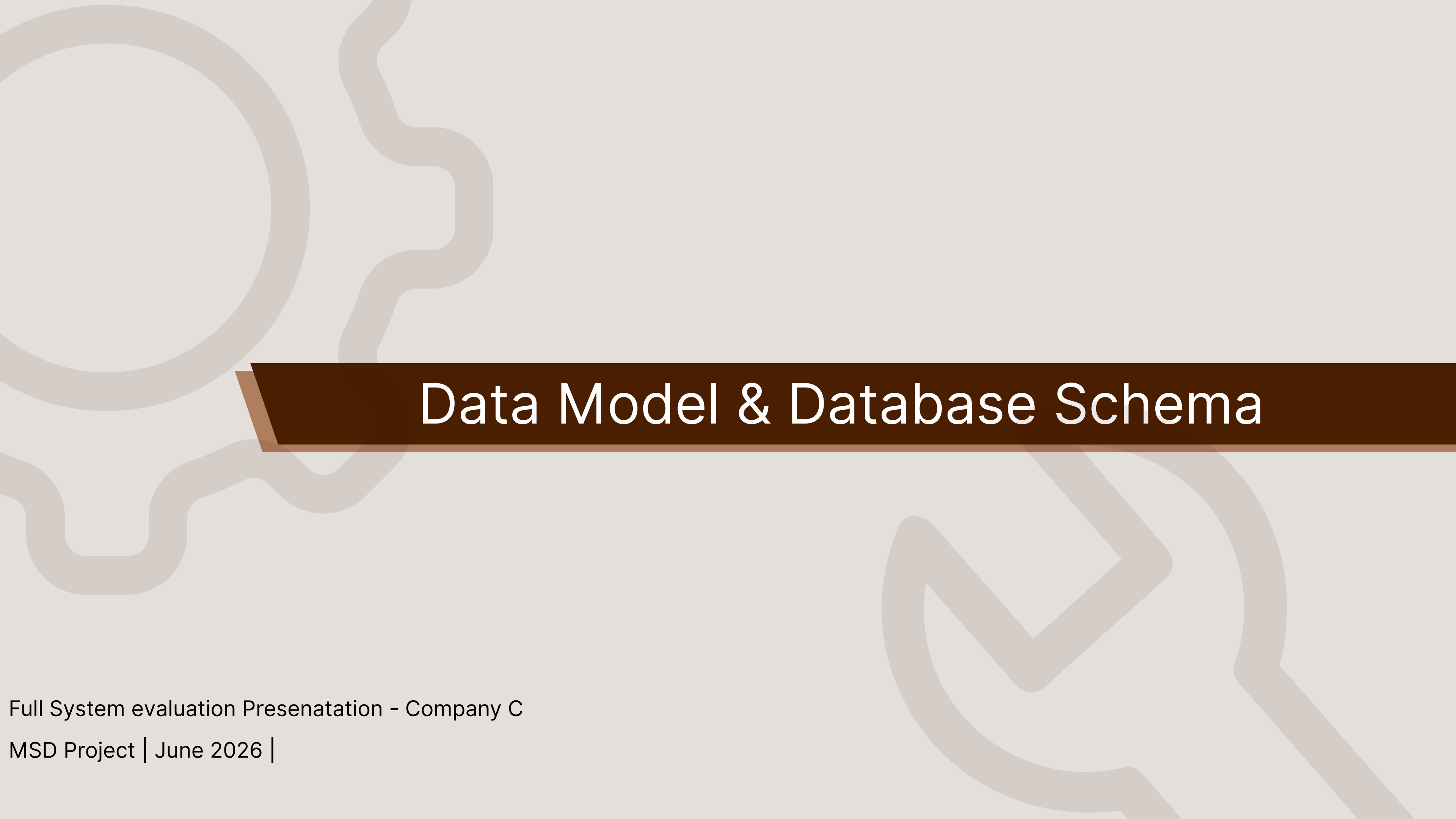
Service

BookingV1Service applies business rules (e.g. rejects blacklisted homeowners), persists via BookingRepository, triggers a notification.

5

Response

Entity mapped back to a DTO, serialised by Jackson. Errors are intercepted by GlobalExceptionHandler into a consistent payload.



Data Model & Database Schema

Full System evaluation Presenatation - Company C
MSD Project | June 2026 |

User (users)

Single table for all roles. user_type ENUM

Service / Category

Service *:1 Category. Base price . Admin can approve new categories via CategoryRequest workflow.

Review (reviews)

@OneToOne Booking. 1–5 rating, comment, isFlagged. Provider avg rating & badge recalculated on each review.

WalletTransaction

Provider withdrawals. Available balance = SUM(completed booking earnings) – SUM(withdrawals).

Booking (bookings)

Central entity. @ManyToOne → homeowner User, provider User, Service. Carries BookingStatus, scheduled_date, address, estimated/final cost.

Payment (payments)

@OneToOne Booking. Stripe PaymentIntent ID, amount, PaymentStatus. Confirmed on Stripe webhook.

Notification / Dispute

Notifications @ManyToOne User, isRead, channel. Disputes raised against bookings (OPEN → RESOLVED / ESCALATED).

RefreshToken / NewsArticle

@ManyToOne User; opaque token, expiry, revoked flag.
NewsArticle: CMS content (admin-managed).



Security & Authentication

Full System evaluation Presenatation - Company C
MSD Project | June 2026 |

BCrypt Passwords

- Never stored in plain text
- BCrypt hashes before persistence
- Password never returned in any DTO
- AuthController nulls hash before response

JWT — Stateless Auth

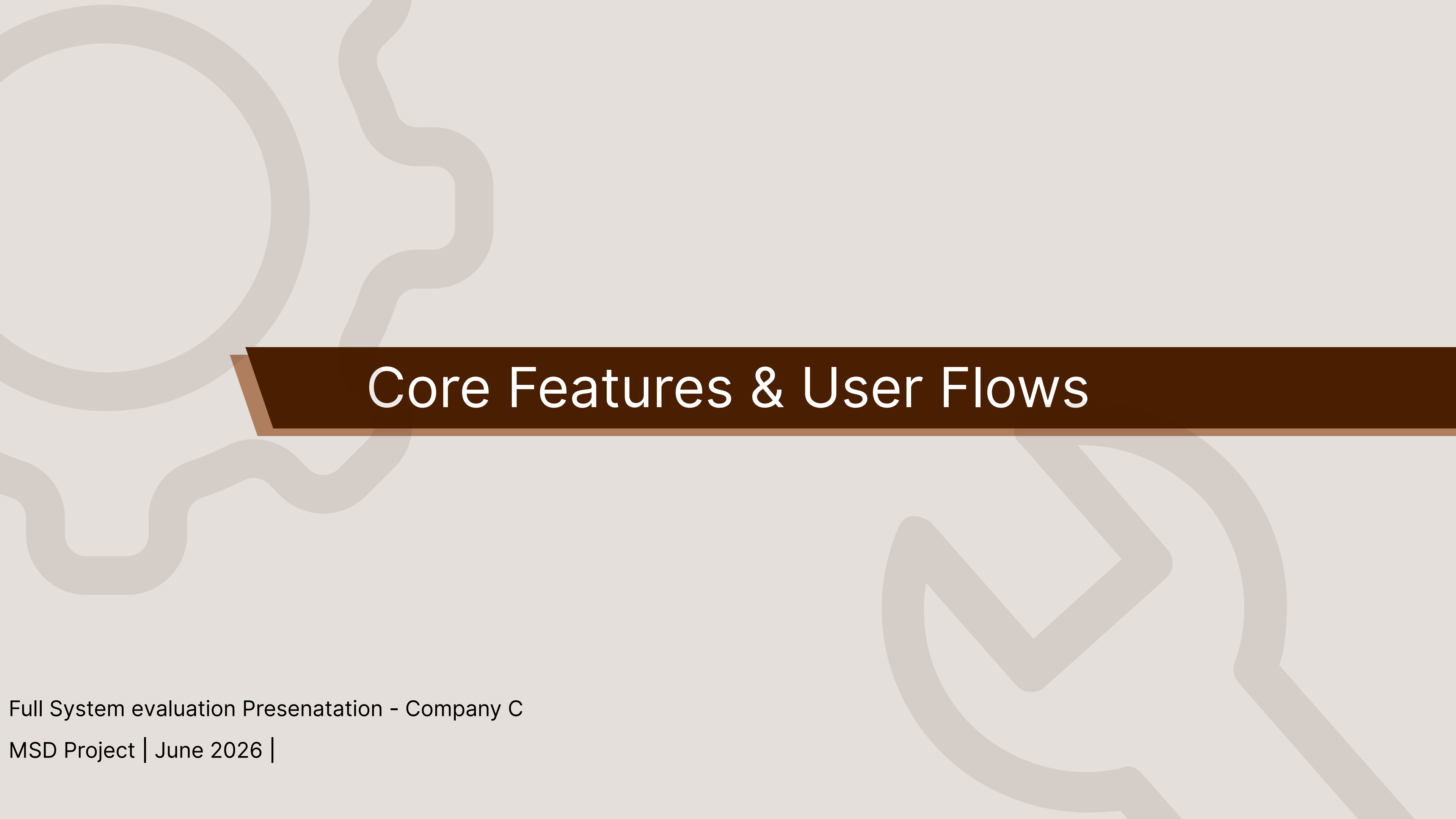
- Login → JwtTokenProvider issues signed access token (HS256) + refresh token
- Client sends Authorization: Bearer <access_token> on every request
- JwtAuthFilter validates → UserPrincipal into SecurityContext
- 401 → frontend transparently refreshes (single-flight guard prevents race)

RBAC — Role-Based Access

- 3 roles: ROLE_HOMEOWNER / ROLE_SERVICE_PROVIDER / ROLE_ADMIN
- @PreAuthorize("hasRole('ADMIN')") on every sensitive endpoint
- Security config matcher set for public vs protected paths
- Provider endpoint → 200; Admin endpoint from provider → 403

Google OAuth2

- Spring Security oauth2Login
- Stateless: auth request stored in cookie (not session)
- OAuth2LoginSuccessHandler: find-or-create user, mint JWT
- App still boots without Google client ID configured



Core Features & User Flows

Full System evaluation Presenatation - Company C
MSD Project | June 2026 |

Registration & Login — Complete Flow

Sign Up (POST /api/v1/auth/signup)

- Validate input, check email/username uniqueness
- Generic duplicate message → prevents account enumeration
- BCrypt-hash password, persist User record
- Provider signup → opens ProviderRequest (starts unverified)
- Returns access token + refresh token + user session object

Login (POST /api/v1/auth/login)

- Spring AuthenticationManager verifies credentials
- Bad credentials → generic 401 (no enumeration leakage)
- POST /api/v1/auth/refresh → new access token
- POST /api/v1/auth/logout → revokes refresh token
- GET /api/v1/auth/session → returns current session
- Google Sign-In → OAuth2 flow → mint own JWT

Booking Lifecycle - Finite State Machine

PENDING



ACCEPTED



IN PROGRESS



COMPLETED

Key Endpoints

- POST /api/v1/bookings — create booking
- PATCH .../bookings/{id}/status — advance state
- GET .../bookings/active — homeowner active view
- GET .../bookings/past — booking history
- GET .../bookings/match — smart provider search

Smart Provider Matching & Live Job Tracking

SmartMatchService - Provider Ranking

- Filters: verified=true, active=true, correct category.
- Circle distance between homeowner & provider.
- Weighted score = rating × weight + (1/distance) × weight
- Returns best N matches.

Live Tracking - WebSocket / STOMP

- WebSocketConfig registers STOMP broker at /ws/tracker.
- Homeowner subscribes and sees provider moving on map.
- TrackingService handles concurrent broadcast to many subscribers.
- Stateless backend → safe for horizontal scaling.

Payments, Wallet & Reviews

Stripe Payments*

- Stripe webhook confirms payment → updates PaymentStatus
- Both parties notified on payment success
- @OneToOne: one Payment per Booking

Provider Wallet

- Available balance = earned (completed bookings) – withdrawals
- WalletV1Service: earnings statistics, withdrawal processing

Reviews & Disputes

- After completed, homeowner submits 1–5 star Review + comment
- Provider avg rating and badge level auto-recalculated

**not yet implemented*

Admin Back Office — Full Platform Control

Provider Management

Verify/suspend/reinstate providers. Review ProviderRequest applications. View provider stats and history.

Category Requests

Approve or reject new service categories. Approved category → goes live in catalogue immediately.

Analytics Dashboard

Overview metrics, monthly reports, activity feeds. Real-time system health monitoring.

News CMS

Create/edit/publish/delete news articles with image upload. Public read, admin write.

Homeowner Management

View, search, filter all homeowners. Suspend/reinstate users. View booking history per user.

Review Moderation

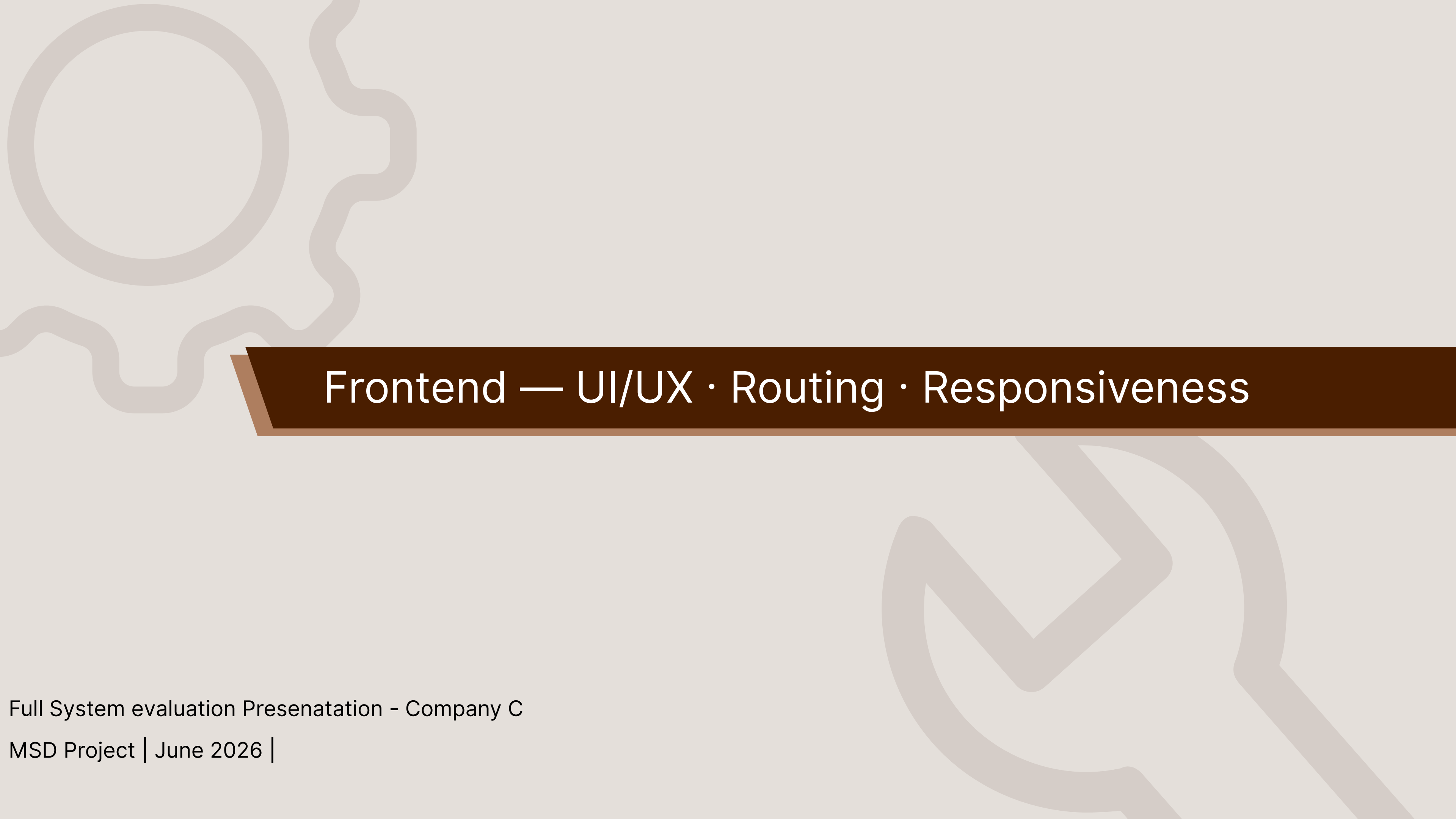
Flag, hide, or resolve flagged reviews. Protect platform trust and rating integrity.

Security Posture

GET /admin/security/events → audit log. GET /admin/system-health → live health check.

Dispute Resolution

View all disputes. Move status OPEN RESOLVED or ESCALATED. Both parties notified.



Frontend — UI/UX · Routing · Responsiveness

Full System evaluation Presenatation - Company C
MSD Project | June 2026 |

Frontend Architecture - React 19 + TypeScript + Vite

Project Structure

- routes/ TanStack file-based routes + auth guards
- pages/ screen-level components
- (HomeownerDashboard, Provider Dashboard,
- components/reusable UI atoms & molecules
- services/ typed API clients (bookings.ts, wallet.ts, providers.ts)
- lib/ api-client.ts, auth-api.ts, booking.ts, http.ts
- hooks/ useAuth, useCurrentUser, custom hooks

Key Architectural Decisions

- File-based routing
- `_authenticated/*` route wraps all protected pages
- - redirects to `/login` if no token
- Feature flag `VITE_USE_NEW_API`: controlled migration to Spring Boot backend
- TanStack Query: server state cache, background refresh, loading/error states
- Single `api-client.ts` with axios: one place to attach
- JWT and handle 401 refresh

Home Module

Files:

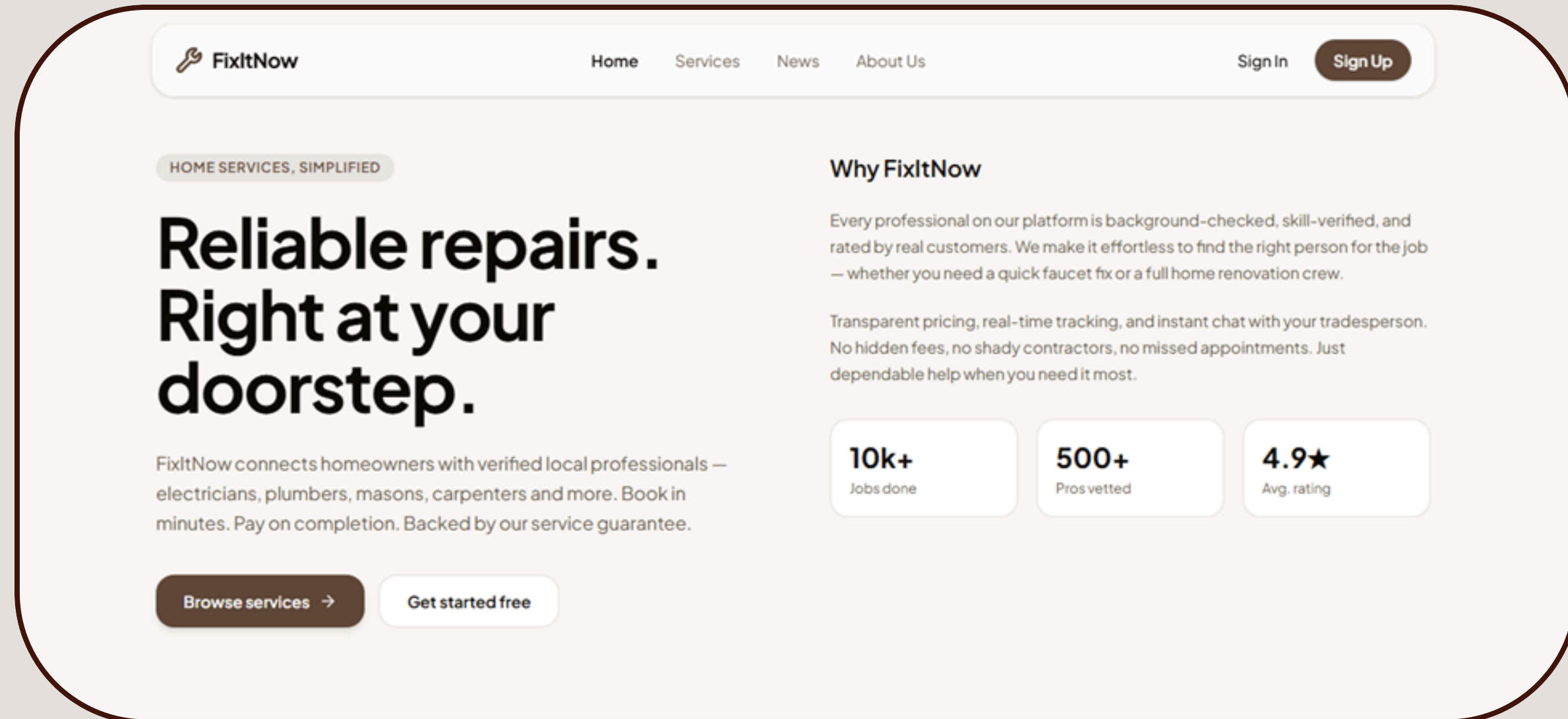
src/pages/home/
src/components/common/

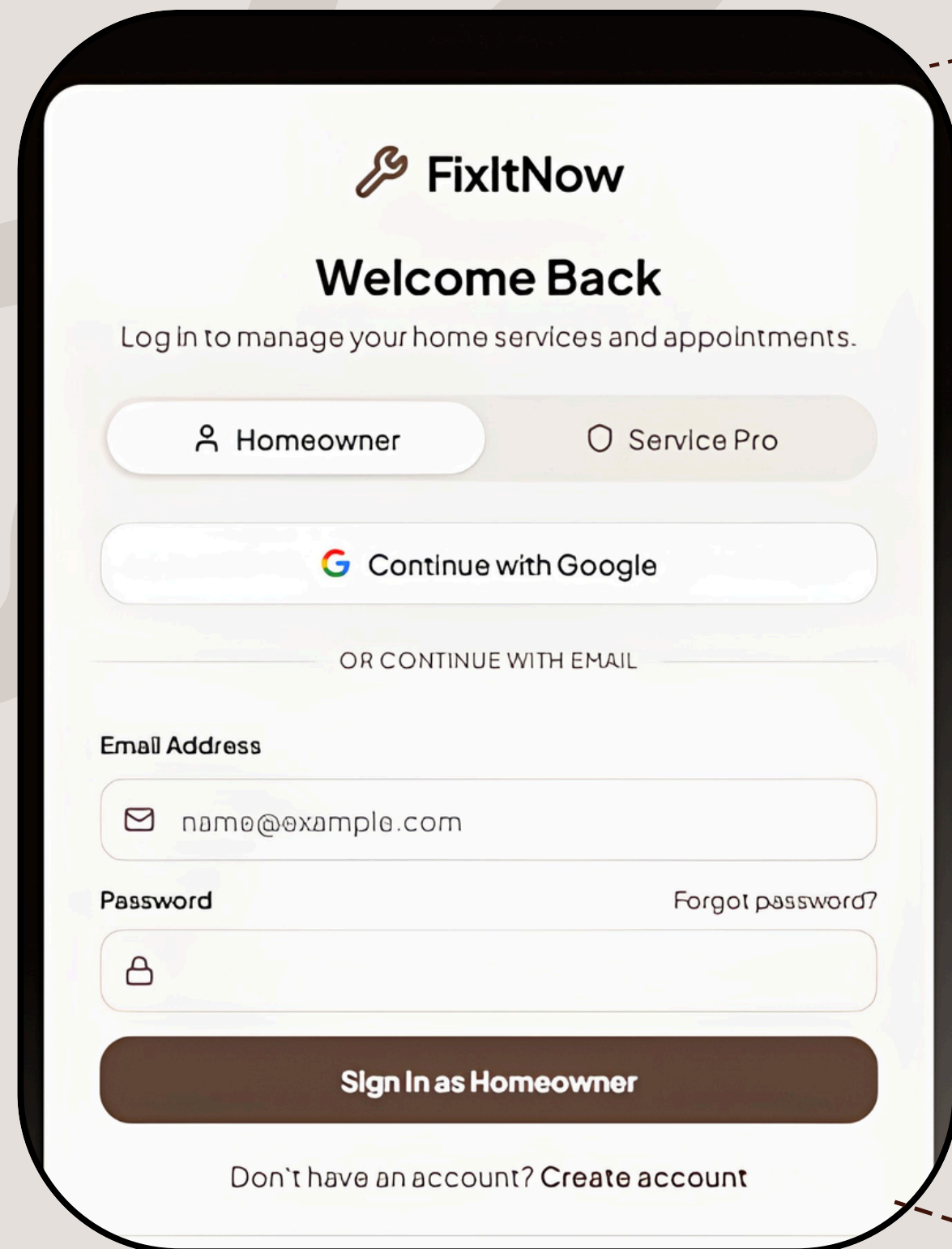
Features:

- Landing page
- Service categories
- Hero section
- Navigation
- Footer

Services displayed:

- Electrician
- Plumber
- Mason
- Carpenter
- Painter
- HVAC
- Welder
- Cleaner





A mobile app login screen for 'FixItNow'. At the top is the logo and 'Welcome Back' heading. Below is a sub-header 'Log In to manage your home services and appointments.' Two role selection buttons, 'Homeowner' (selected) and 'Service Pro', are shown. A 'Continue with Google' button follows. Below a separator line, it says 'OR CONTINUE WITH EMAIL'. There are input fields for 'Email Address' (containing 'name@example.com') and 'Password' (with a 'Forgot password?' link). A large 'Sign In as Homeowner' button is at the bottom, with a link 'Don't have an account? Create account' below it.

FixItNow

Welcome Back

Log In to manage your home services and appointments.

☒ Homeowner ☐ Service Pro

 Continue with Google

OR CONTINUE WITH EMAIL

Email Address

 name@example.com

Password [Forgot password?](#)



Sign In as Homeowner

Don't have an account? [Create account](#)

Authentication Module

Files:

src/routes/login.tsx

src/routes/signup.tsx

src/integrations/supabase/

Features:

- Login
- Registration
- Session handling
- Route protection

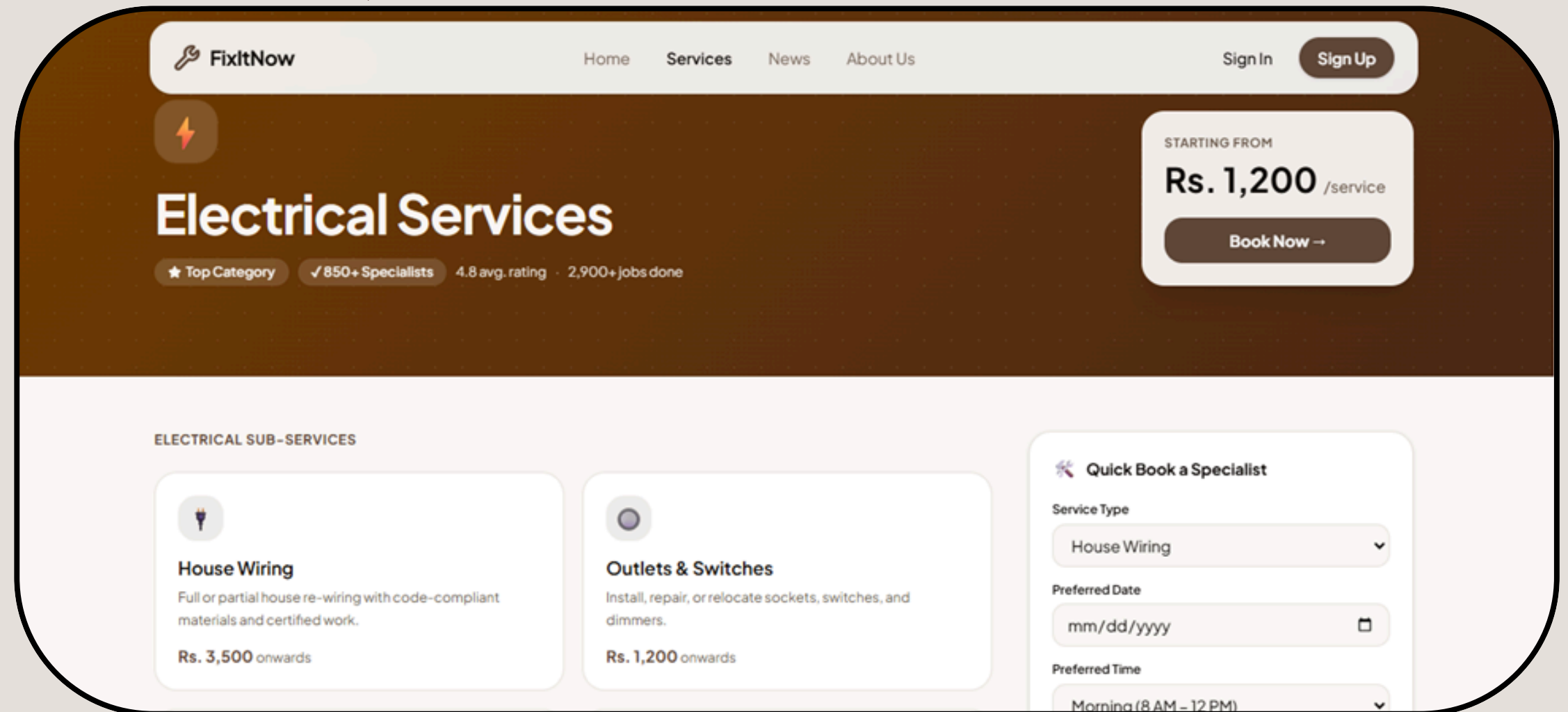
Services Module

Files:

src/pages/services/

Features:

- Browse categories
- View service details
- View sub-services
- Provider selection
- Pages:
 - ServicesPage
 - ServiceDetailPage
 - SubServiceDetailPage



Booking System

One of the most important modules.

Booking Workflow

Step 1

Select Service



Step 2

Choose Provider



Step 3

Schedule Time



Step 4

Review & Payment



Step 5

Booking Confirmation

Main file:

`src/pages/booking/BookingWizard.tsx`

The screenshot displays the 'FixItNow' booking interface. At the top, a navigation bar includes the logo, links for Home, Services, News, and About Us, and buttons for Book Now, Dashboard, and a user profile (JS). Below the navigation bar is a progress indicator with five steps: Select Service (checked), Choose Provider (checked), Schedule (active), Confirm & Pay, and Booking Done. The main heading for the current step is 'When should Janindu arrive?' with a subtext 'Pick a date, time slot, and confirm your address'. The interface is divided into two main sections. The left section, titled 'SELECT A DATE', shows a calendar for June 2026. The right section, titled 'Booking summary', provides details about the booking.

June 2026						
Sun	Mon	Tue	Wed	Thu	Fri	Sat
	1	2	3	4	5	6
7	8	9	10	11	12	13

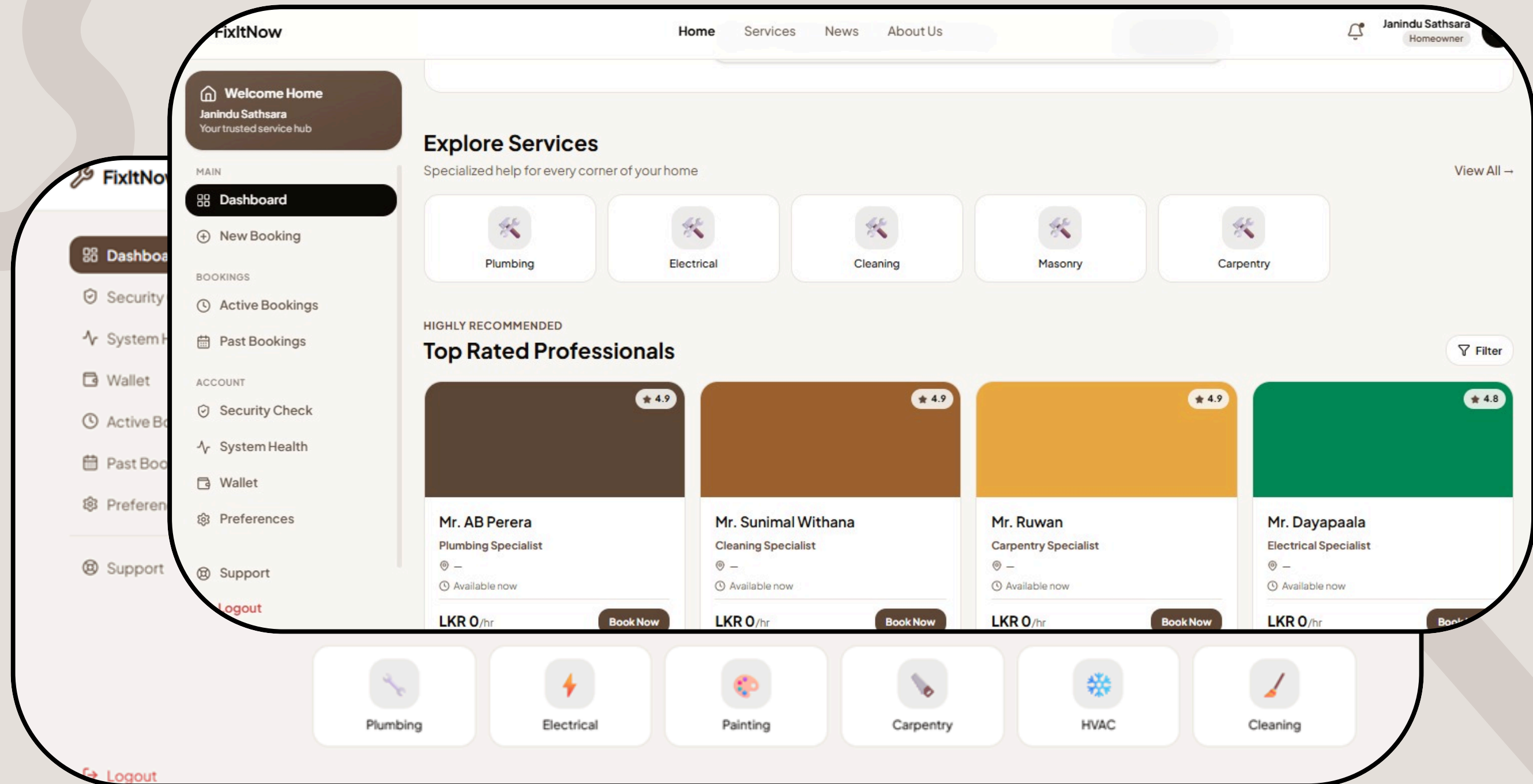
Booking summary	
	Janindu Sathara (Provider) ★ 4.9 · Master Plumber
Service	Faucet & Tap Repair
Date	Wed, Jun 17
Time	10:00 AM
Address	—
Provider Rate	Rs. 2,800/hr
Est. Duration	1-2 hrs
Platform Fee	Rs. 200

Homeowner Dashboard

Files: src/pages/dashboard/HomeownerDashboard.tsx

Features:

- Active bookings
- Service discovery
- Notifications
- Booking history



Provider Dashboard

Files: src/pages/dashboard/ProviderDashboard.tsx

Features:

- Job requests
- Booking management
- Service management
- Reviews
- Wallet

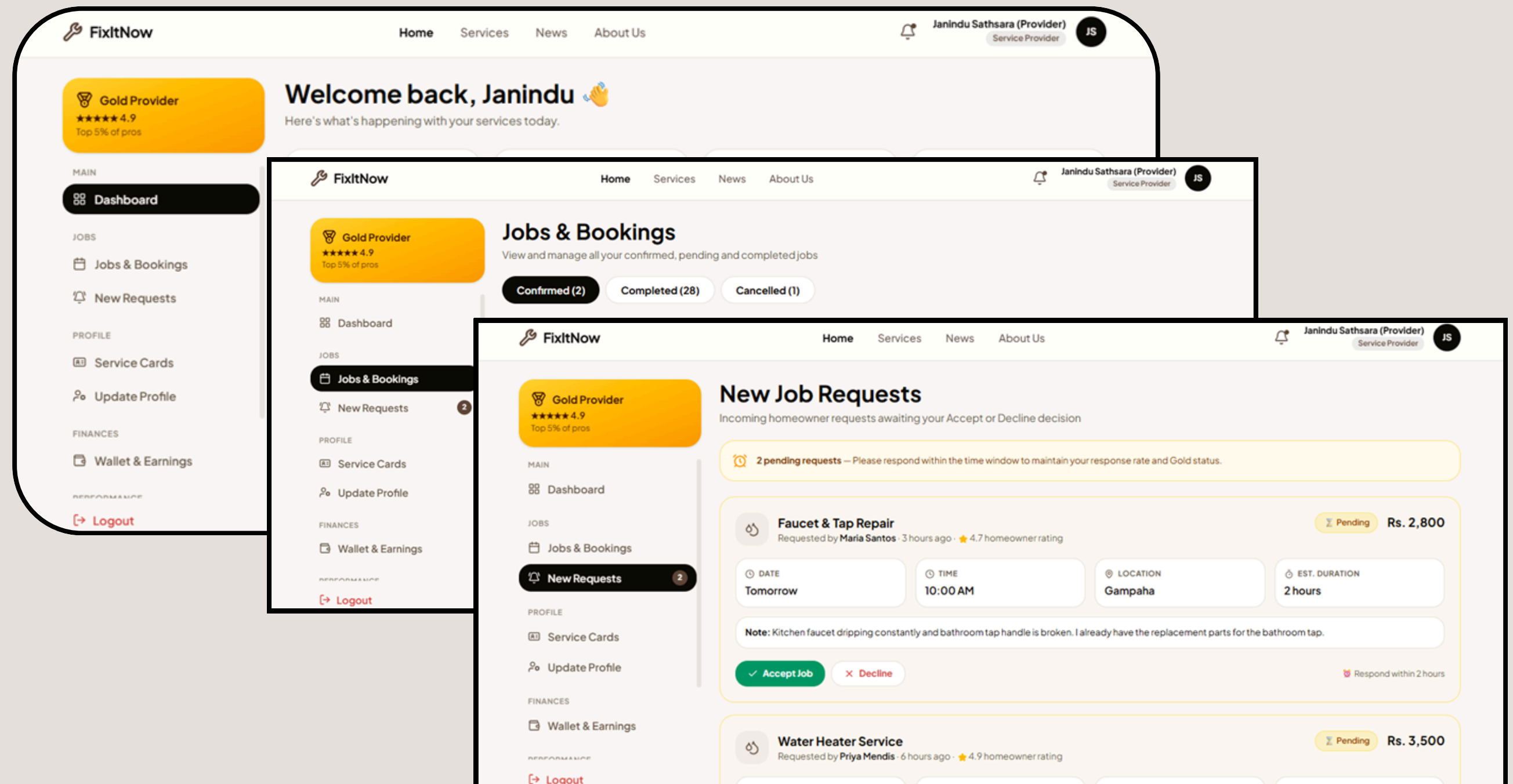
Additional pages:

ProviderJobsPage

ProviderReviewsPage

ProviderWalletPage

ProviderProfilePage



HomeServicesNewsAbout Us

Janindu Sathsara (Provider)

Service Provider

JS

Gold Provider

★★★★★ 4.9

Top 5% of pros

MAIN

Dashboard

JOBS

Jobs & Bookings

New Requests2

PROFILE

Service Cards

Update Profile

FINANCES

Wallet & Earnings

PERFORMANCE

Logout

Service Cards

Manage your public marketplace listings visible to homeowners

Create New Card

Emergency Plumbing

Rs. 2,800 / hr · Min Rs. 1,500

24/7 emergency plumbing for burst pipes, leaks, blockages. Arrives within 60 min. 30-day guarantee.

Published

Edit

Faucet & Tap Repair

Rs. 1,800 / job · Fixed price

Full faucet repair or replacement. Kitchen, bathroom, garden taps. 14-day workmanship

Pipe Replacement

Rs. 3,200 / hr + materials

Complete pipe inspection, repair or replacement. UPVC, copper and PVC. Pressure

HomeServicesNewsAbout Us

Janindu Sathsara (Provider)

Service Provider

JS

Gold Provider

★★★★★ 4.9

Top 5% of pros

MAIN

Dashboard

JOBS

Jobs & Bookings

New Requests2

PROFILE

Service Cards

Update Profile

FINANCES

Wallet & Earnings

PERFORMANCE

Logout

Wallet & Earnings

Track your income, initiate payouts and manage your financial account

AVAILABLE BALANCE

Rs. 124,800

Last updated today · Pending: Rs. 12,700 in escrow

Transfer to BankHistoryStatement

THIS WEEK

Rs. 18,400

↑ 18% vs last week

THIS MONTH

Rs. 84,200

↑ 12% vs May

TOTAL (YTD)

Rs. 842,000

↑ 22% vs 2025

AVG. PER JOB

Rs. 6,578

↑ Rs. 320 vs last month

Weekly Earnings — Last 8 Weeks

Transfer to Bank

PAYOUT AMOUNT (RS.)

Admin Module

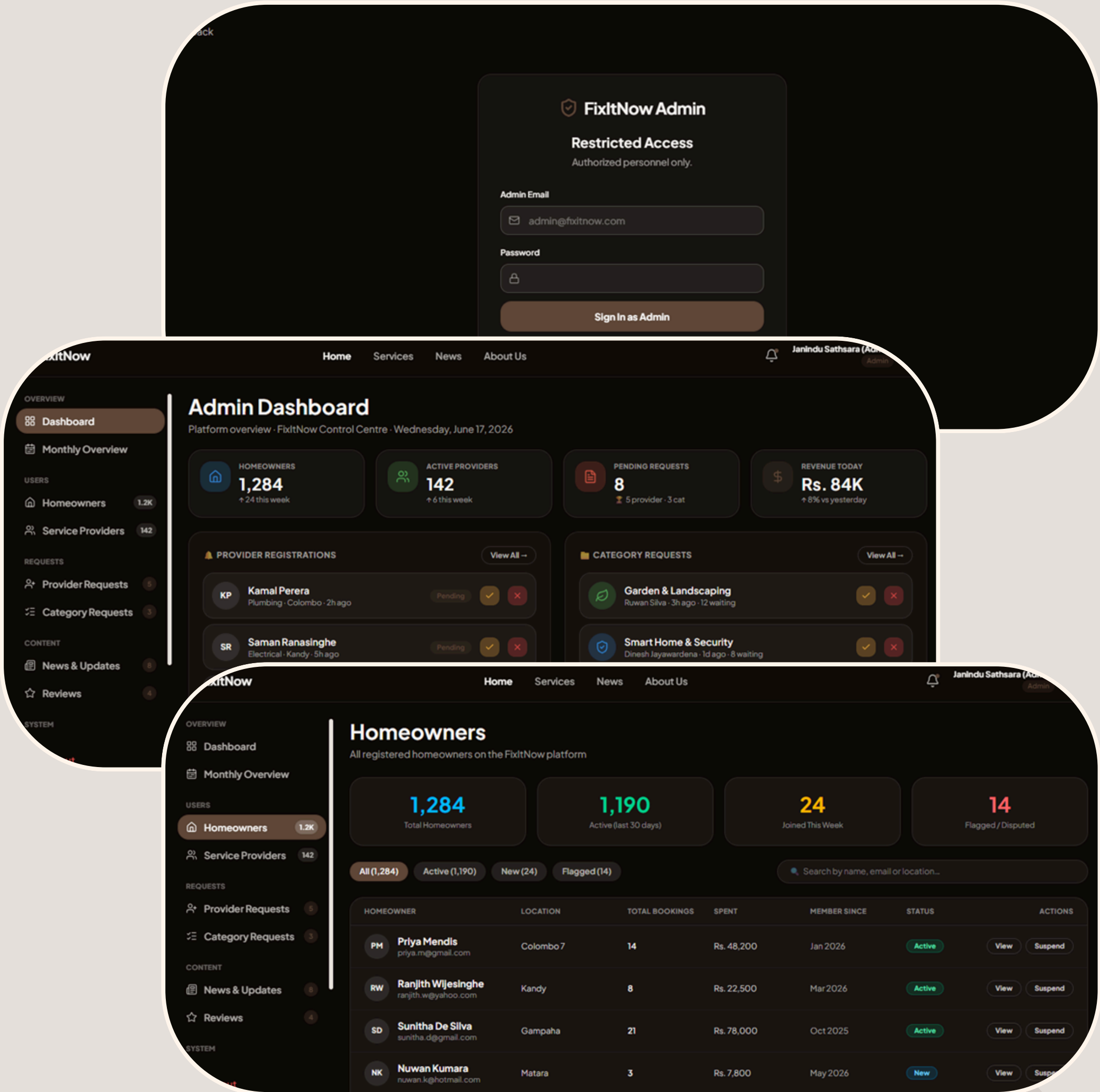
Admin Module Files: src/pages/admin/

Admin capabilities:

- User Management
- Providers
- Homeowners Content Management
- News articles
- Announcements Monitoring
- Security dashboard
- System health
- Monthly reports

Pages include:

AdminProvidersPage
AdminHomeownersPage
AdminNewsPage
AdminSecurityPage
AdminSystemHealthPag



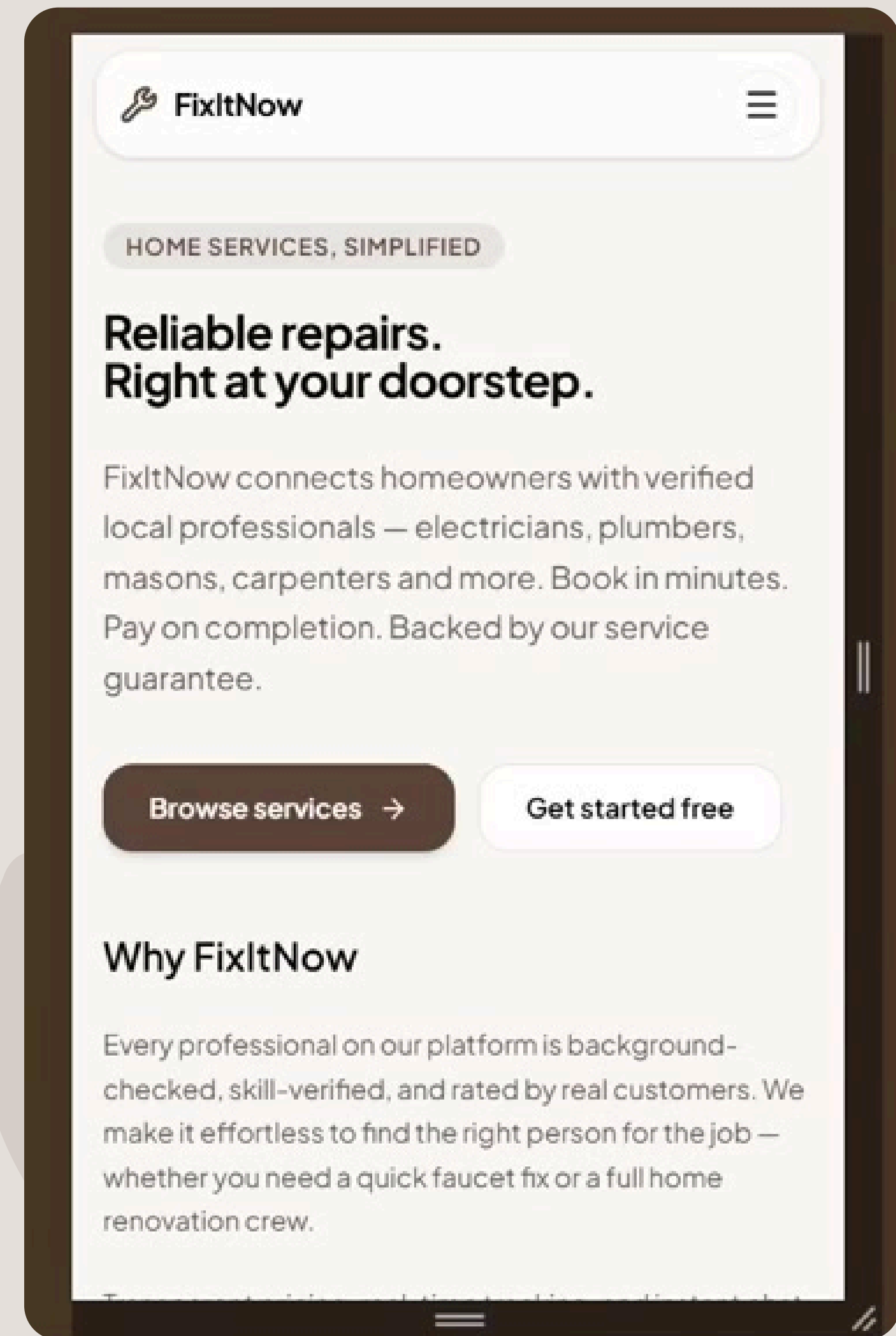
Mobile Responsiveness — Tailwind CSS Utility-First

Responsive Design Techniques

- Tailwind sm:/md:/lg: breakpoint prefixes on every component
- Flexbox + CSS Grid for fluid layouts
- Stack on mobile, side-by-side on desktop
- Touch-friendly tap targets (min 44x44px)
- Navigation collapses to hamburger on mobile
- Cards/grids reflow: 1-col → 2-col → 3-col

Component Design Principles

- Radix UI primitives: accessible, keyboard-navigable
- React Hook Form: optimised re-renders, no janky forms
- TanStack Query: skeleton loaders, error boundaries
- Recharts: responsive container wrapper for charts
- Consistent spacing/typography via Tailwind tokens
- Dark header, light content (sandwich design)



Routing Structure

>> The project uses file-based routing.

Examples:

/

|—— Home

|—— /services

|—— /login

|—— /signup

|—— /news

|—— /profile

|—— /book

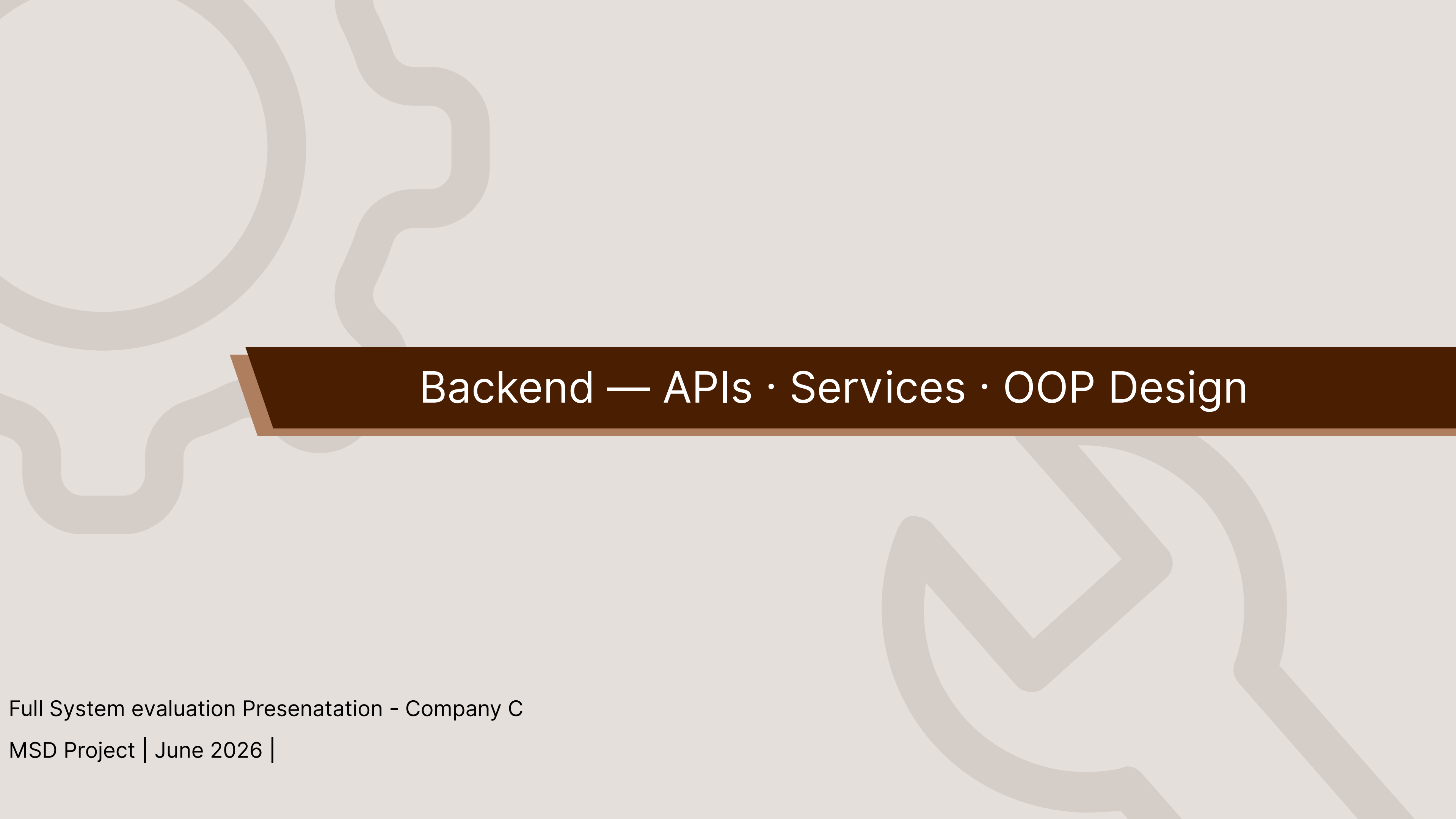
|—— /dashboard/homeowner

|—— /dashboard/provider

|—— /dashboard/admin

Protected routes:

/_authenticated/*



Backend — APIs · Services · OOP Design

Full System evaluation Presenatation - Company C
MSD Project | June 2026 |

REST API Reference - Key Endpoints

Auth (public)

POST /api/v1/auth/signup · /login · /refresh · /logout
GET /session · /oauth/google

Bookings (homeowner)

POST /api/v1/bookings — create
PATCH .../bookings/:id/status — state transition
GET .../bookings/active · /past · /match

Provider Self

GET /providers/me/stats · /reviews · /wallet/*
PATCH /providers/me — profile update

News CMS

GET /api/v1/news — public read
POST/PATCH/DELETE /api/v1/news — admin write
POST /api/v1/news/:id/image — image upload

Browse (public)

GET /api/v1/service-categories
GET /api/v1/service-categories/:id/sub-services
GET /api/v1/providers?category_id=&available=true

Profile (auth)

GET/PATCH /me — own profile
GET /homeowners/me/stats — activity stats

Admin (ADMIN role)

GET/PATCH /providers · /homeowners · /reviews
GET /dashboard/* · /security/events · /system-health
POST/PATCH/DELETE /provider-requests · /category-requests

Disputes

POST /api/v1/disputes — raise dispute
PATCH /api/v1/disputes/:id — admin resolve

OOP Design - Code Evidence

Encapsulation

AuthSessionV1: private final fields, @Builder, no setters

AuthController nulls password hash before returning. Response DTOs immutable after construction.

Interfaces

UserRepository extends JpaRepository<User, Long>

CustomUserDetailsService implements UserDetailsService.

Collections

LinkedHashMap in adapters → stable JSON field order

List for ordered results, Set for OAuth scopes/authorities

Inheritance

JwtAuthFilter extends OncePerRequestFilter

Custom exception hierarchy

Generics

ApiResponse<T> with static <T> ApiResponse<T> ok(...)

Type-safe response envelope. Optional<User>, List<Booking>, ResponseEntity<List<CategoryV1>>

Concurrency

@Async NotificationService.sendEmail()

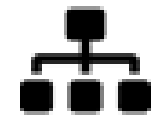
STOMP WebSocket concurrent broadcast.

Eight OOP Pillars, Mapped to Real Code



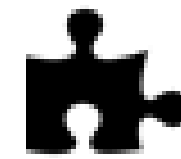
Encapsulation

Immutable DTOs (private final + @Builder); password hash never returned.



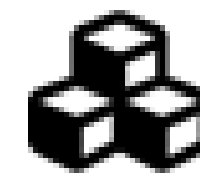
Inheritance

Exception hierarchy extends RuntimeException; framework extension points override template methods.



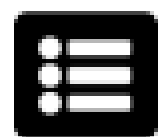
Interfaces

14 repositories as pure interfaces;
CustomUserDetailsService implements
UserDetailsService



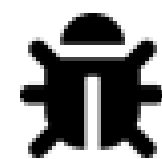
Generics

ApiResponse<T> with generic static factories;
JpaRepository<User, Long>.



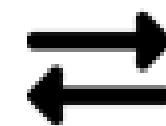
Collections

List for ordered results,
LinkedHashMap for stable JSON field order,
Set for uniqueness



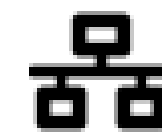
Exception Handling

@RestControllerAdvice
GlobalExceptionHandler maps each type to HTTP status + ApiResponse.



Concurrency

@Async email dispatch,
STOMP broadcast, single-flight token refresh,
stateless scaling



Serialization

Jackson JSON with @JsonProperty contracts;
ObjectOutputStream + Base64 for stateless OAuth



Testing and Quality deliverables

Full System evaluation Presenatation - Company C
MSD Project | June 2026 |

A Layered Testing Strategy



Unit

JUnit 5 + Mockito

@InjectMocks / @Mock,
AuthServiceTest,
BookingServiceTest,
SmartMatchServiceTest.



Integration

@SpringBootTest +
MockMvc

FlowIntegrationTest
drives real HTTP routing
with @MockitoBean
stubs; asserts via
jsonPath.



Security / RBAC

@WithMockUser

Role-based access
exercised with status-
code assertions across
the integration suite.



Acceptance

UserAcceptanceTest

Scenario-level
validation of five end-
to-end workflows.



End-to-End

Live MySQL + real
HTTP

Manual verification: role
propagation, OAuth
redirect chain, stateless
cookies.

Representative Test Cases

Unit Test — Negative Path

```
@Test
@DisplayName("register() -- duplicate email
            should throw BadRequestException")
void ut02_register_duplicateEmail() {
    // Arrange
    when(userRepository.existsByEmail(email))
        .thenReturn(true);
    // Act + Assert
    assertThatThrownBy(() ->
        authService.register(validRequest))
        .isInstanceOf(BadRequestException.class);
    verify(userRepository, never()).save(any());
}
```

Integration Test — Routing

```
@Test
@WithMockUser(username="alice",
roles="HOMEOWNER")
@DisplayName("IT-01 | Search provider, book")
void it01_searchProviderThenBook() {
    when(smartMatchService.findMatches(
        "PLUMBING", 6.9271, 79.8612))
        .thenReturn(List.of(match));

    mockMvc.perform(get("/api/bookings/match")
        .param("category", "PLUMBING"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.success").value(true));
}
```

Security & Performance Testing

Security Testing

Security testing focuses on protecting the FixItNow platform from common web application vulnerabilities.

Key test cases:

- Password encoding
- SQL injection
- XSS prevention
- Role-based access control
- Session security

Performance Testing

Performance testing ensures the FixItNow platform meets the SRS requirements for speed, scalability, and stability.

Key test cases:

- 500 concurrent users
- Response under 2 seconds
- Memory usage (no memory leaks)
- Stress test

Regression Testing

The **Regression testing process** is mainly focused on re-running existing tests after code changes to ensure previously working functionality is not broken.

In Fix-It Now this directly maps to FR12: "Ensure updates don't break features , Notifications still work after updates" and was extended to cover other core features of the platform.

Regression Test Suites

8 test suites were written, covering 160 individual test cases across the platform's core features:

- AuthRegressionTest - 17 tests
- BookingRegressionTest - 24 tests
- PaymentRegressionTest - 18 tests
- ReviewRegressionTest - 18 tests
- SmartMatchRegressionTest - 18 tests
- UserManagementRegressionTest - 24 tests
- DisputeRegressionTest - 24 tests
- NotificationRegressionTest - 17 tests

API Testing

Tool: **Postman Collection Runner**

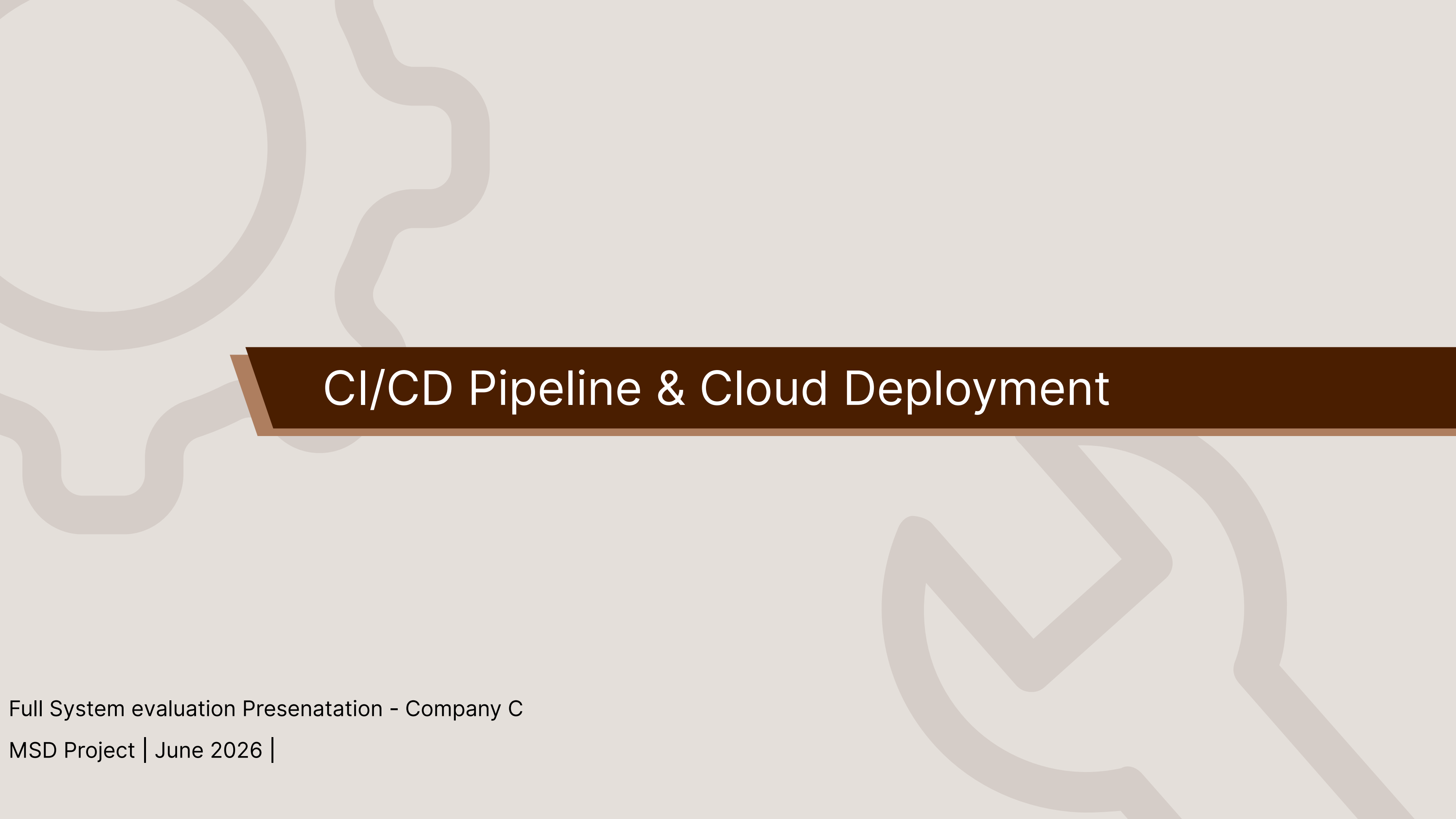
Collection Overview:

- 15 folders, 84 requests covering all backend endpoints
- Automated test scripts on every request
- Bearer token auto-saved after Login and reused across all requests
- Negative testing included (401, 403, 400)

Folders Tested:

- 1.Auth
- 2.Profile (Me)
- 3.Service Catalog
- 4.Bookings
- 5.Notifications
- 6.News
- 7.Reviews
- 8.Provider – Self Service
- 9.Wallet

- 10.Provider Requests
- 11.Category Requests
- 12.Admin – Homeowners
- 13.Admin – Providers
- 14.Admin – Dashboard & System
- 15.Service Cards



CI/CD Pipeline & Cloud Deployment

Full System evaluation Presenatation - Company C
MSD Project | June 2026 |

AWS Cloud Deployment - Production Environment

Live at: <https://fixitnow.chanupahansaja.me> | AWS EC2 44.203.20.87 | Amazon Linux 2023

AWS EC2 Instance

Amazon Linux 2023 VM. 1 GB RAM, swap configured. Hosts both the Spring Boot backend and the React SPA static files.

nginx (Reverse Proxy)

Terminates TLS, serves static React SPA files.

Let's Encrypt TLS

Free, auto-renewing SSL certificate via certbot. HTTPS enforced site-wide. Certificate auto-renewed on expiry.

systemd Service

fixitnow.service manages the Spring Boot JVM process. Restarts on failure. Memory-tuned JVM flags for 1 GB box.

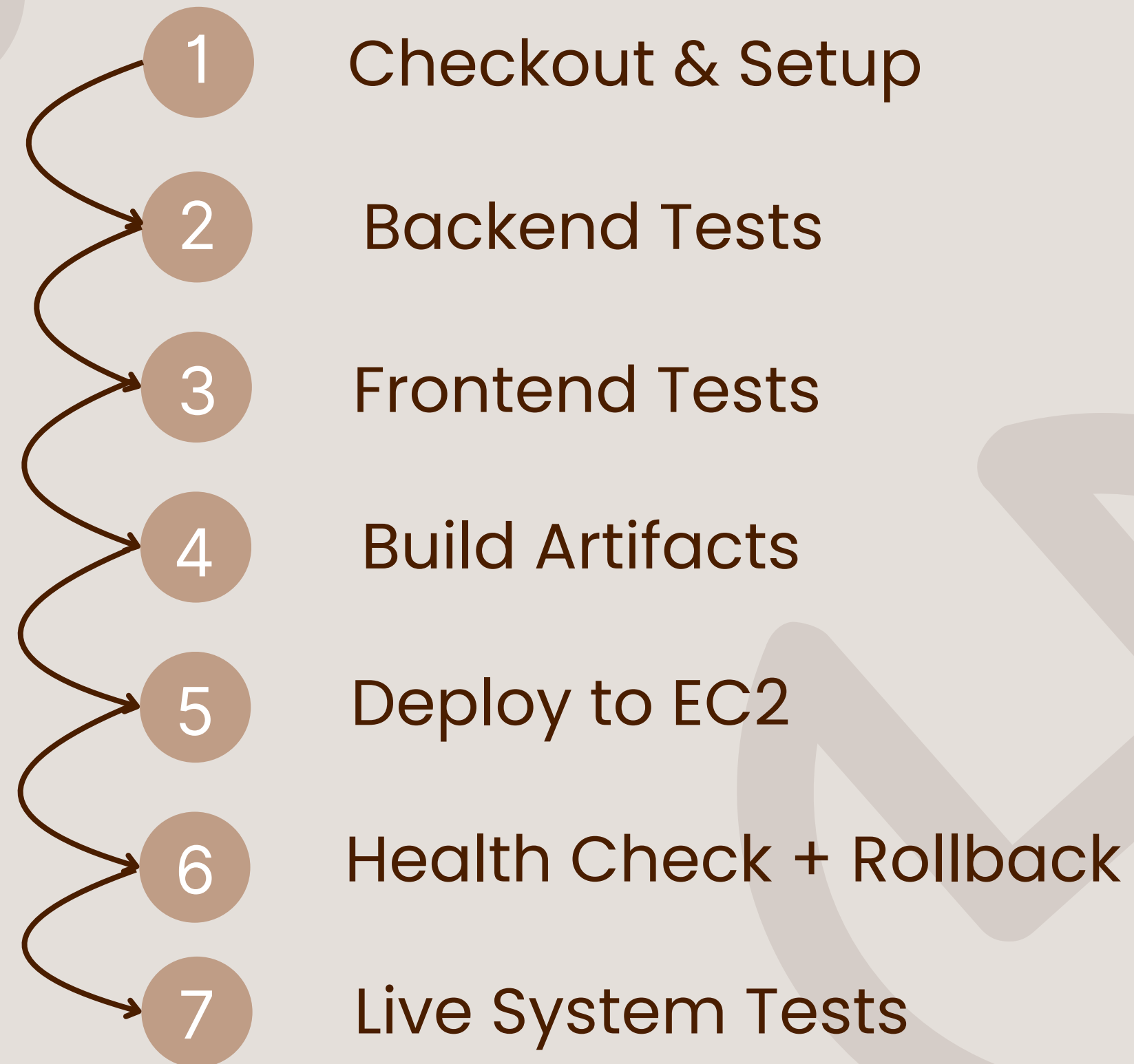
MariaDB

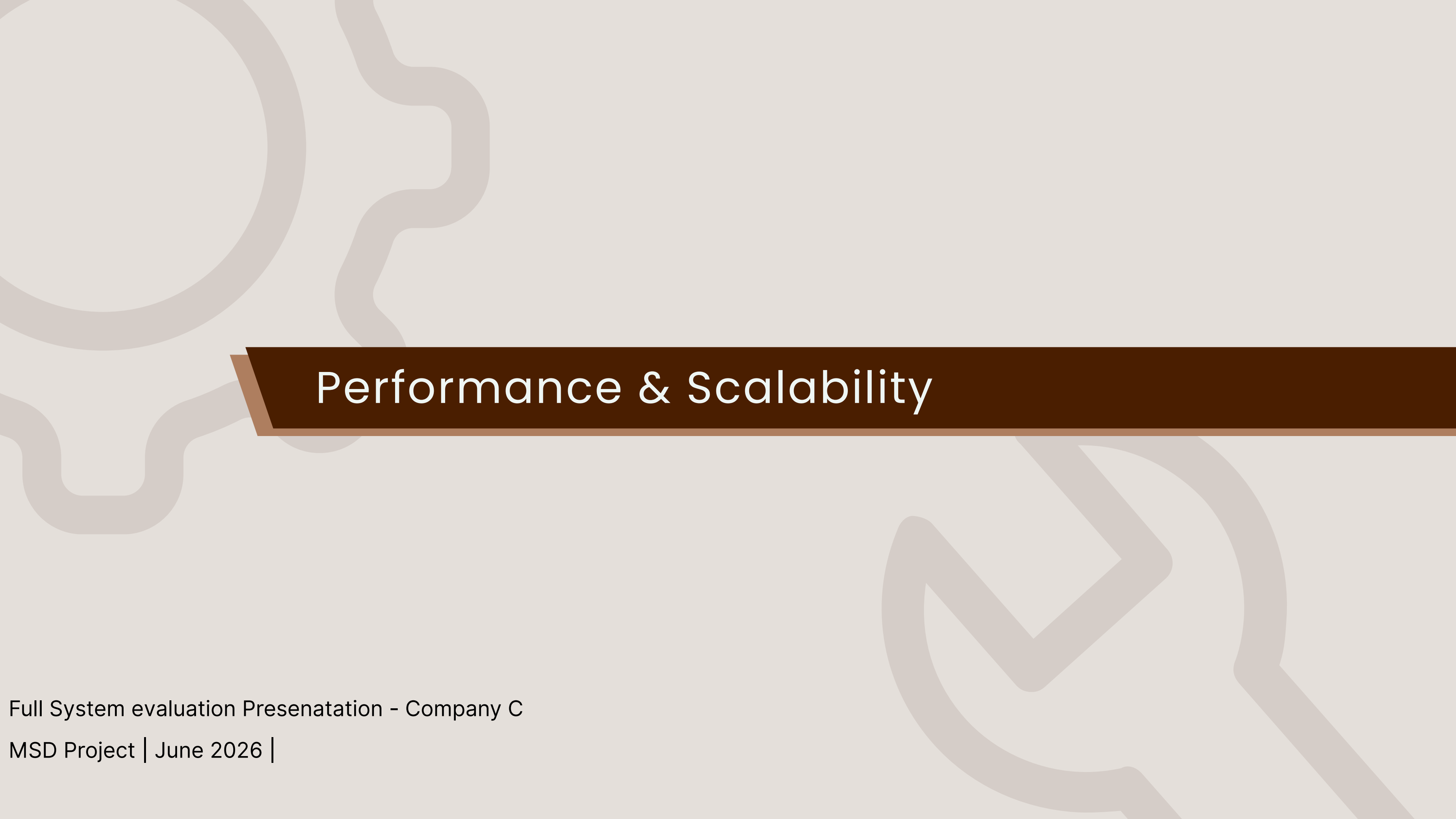
Production database engine (MySQL-compatible). fixitnow_db database. Data persisted on EC2 EBS volume.

Security Hardening

SSH with pinned known_hosts. No plain-text secrets in JAR (CI guard checks this). RBAC on all admin endpoints.

CI/CD Pipeline - GitHub Actions (Test → Deploy)





Performance & Scalability

Full System evaluation Presenatation - Company C
MSD Project | June 2026 |

Performance, Scalability & Non-Functional Qualities

Stateless Backend

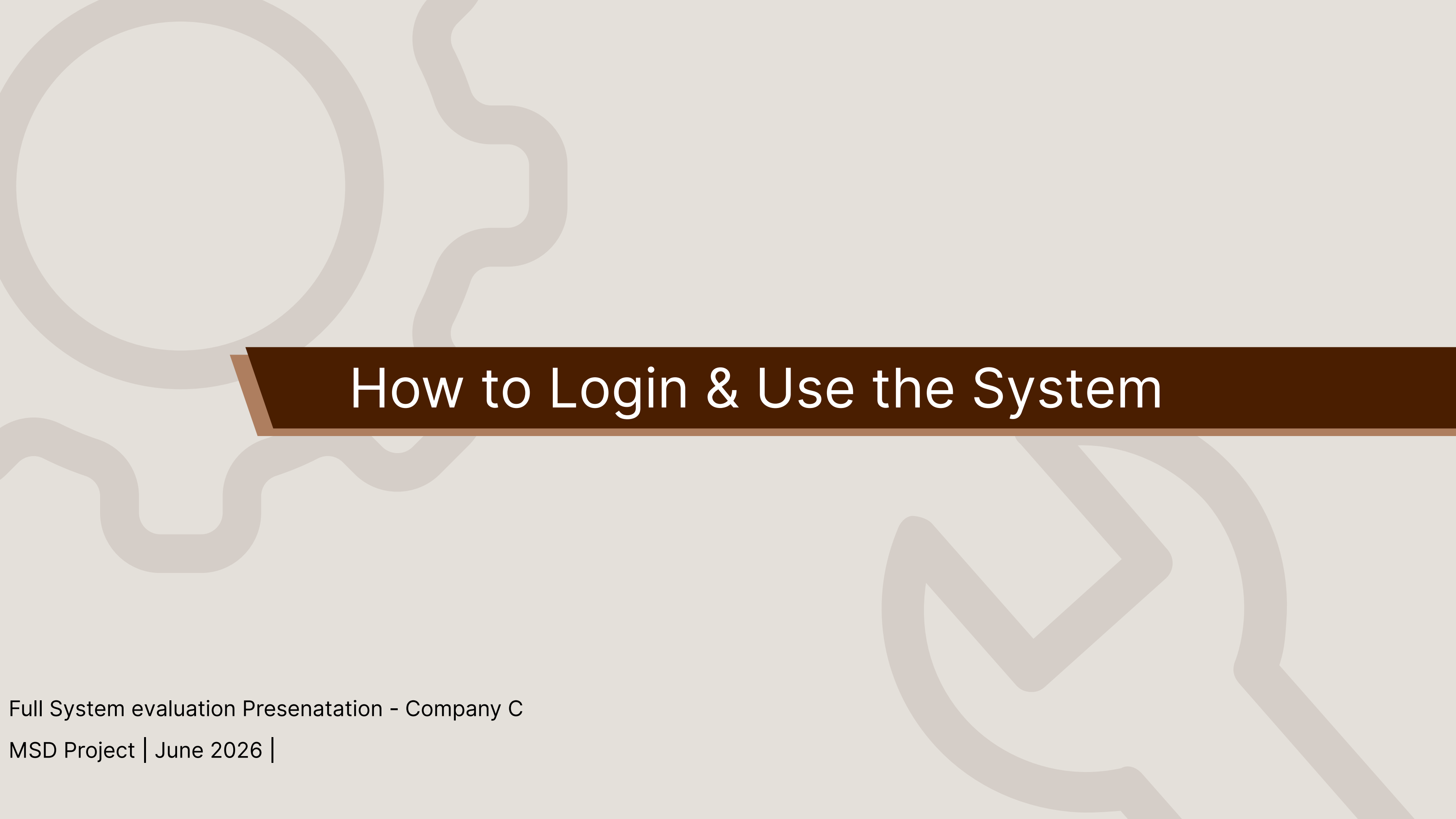
- Session Creation Policy - no session storage
- Each request carries its own identity in JWT

Frontend Performance

- TanStack Query: intelligent caching, background refetch

Database Efficiency

- Spring Data JPA with indexed foreign keys



How to Login & Use the System

Full System evaluation Presenatation - Company C
MSD Project | June 2026 |

How to Login - Step-by-Step Guide

1

Open the App

Visit <https://fixitnow.chanupahansaja.me> in any browser. Click 'Sign In' (top-right) or 'Get started free' on the hero banner.

2

Choose Login Method

Option A: Email + Password — enter registered email and password, click Login. Option B: 'Sign in with Google' button — Google OAuth flow redirects back automatically with JWT tokens.

3

Registration (New User)

Click 'Sign Up'. Choose role: Homeowner or Service Provider. Fill name, email, password (+ service category for providers). Provider starts 'unverified' pending admin review.

4

Token Storage

On successful login: access token + refresh token stored in browser. All subsequent API calls include Authorization: Bearer <token>. Protected routes auto-redirect to /login if no token.

5

Session Refresh

Access tokens expire automatically. Frontend transparently calls POST /api/v1/auth/refresh. User never sees a logout unless refresh token also expires.

6

Logout

Click user menu → Logout. Calls POST /api/v1/auth/logout → revokes refresh token on server. Redirect to landing page.

How to Use - Homeowner Complete Journey

Browse Services

Go to /services → Choose a category (Plumbing, Electrical, Painting, HVAC, etc.) → View service details and sub-services

Find a Provider

System calls SmartMatchService → Nearby verified providers ranked by rating + distance. View profiles, ratings, badges.

Book (BookingWizard)

Select service → Choose provider → Schedule date/time → Review quote → Confirm booking. Booking created as PENDING.

Provider Accepts

Provider receives notification → PATCH status → ACCEPTED. You're notified. Provider heads to your location.

Track in Real-Time

Open active booking → WebSocket / STOMP*

Complete & Pay

Provider marks COMPLETED → Stripe PaymentIntent created → Payment confirmed. Both parties notified*.

Leave a Review

Rate 1–5 stars, add comment. Provider's average rating and badge updated. Raise a Dispute if needed.

Project Management & Development Approach

Repository Structure

Two Git repos: backend + frontend. Deployed together on the same EC2 instance.

Branch Strategy

main branch is production. Every push to main triggers CI/CD. PR-based workflow with tests gating merges.

Agile-Inspired Delivery

Feature-by-feature delivery following the domain model. Core auth first, then bookings, payments, admin, notifications, WebSocket.

Database Migrations

db/schema.sql + db/seed.sql. Hibernate generates schema from JPA entities. Separate seed data for development.

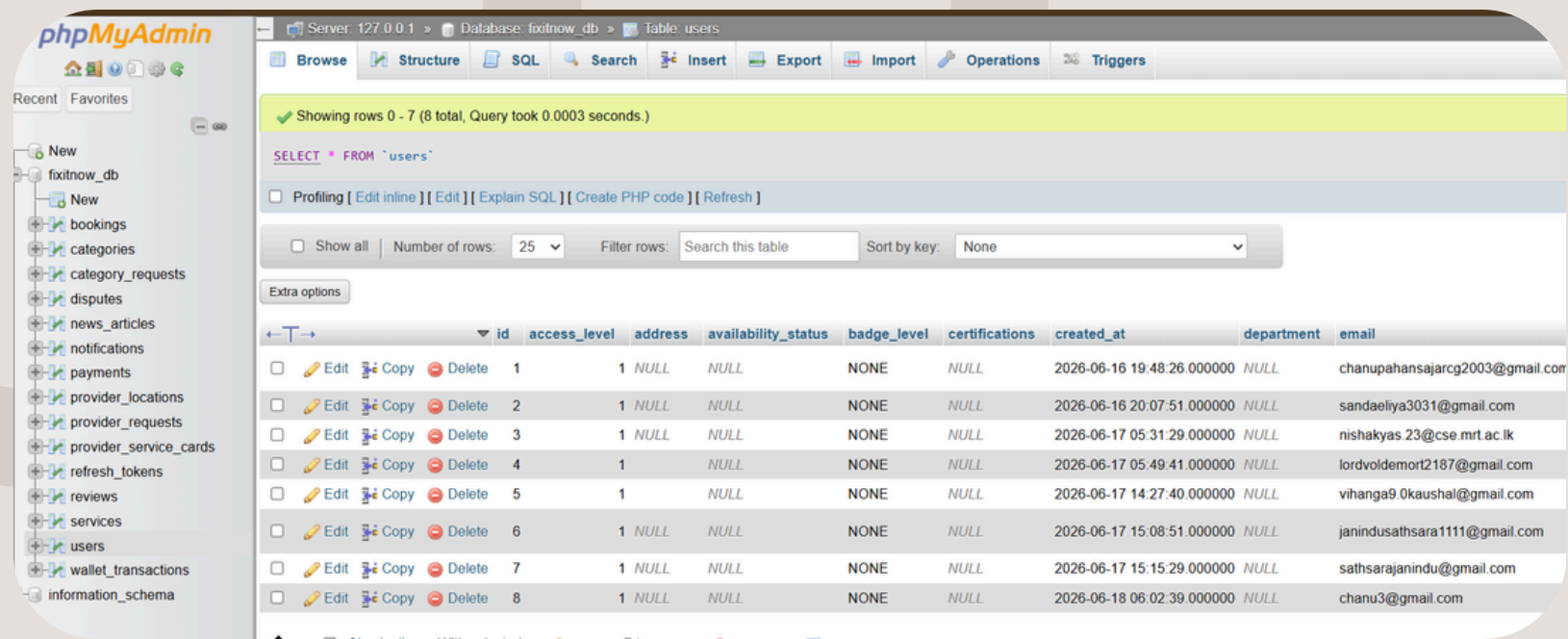
Environment Management

application-local.properties excluded from JAR (CI guard). VITE_* env vars for frontend. Secrets in GitHub Actions Secrets.

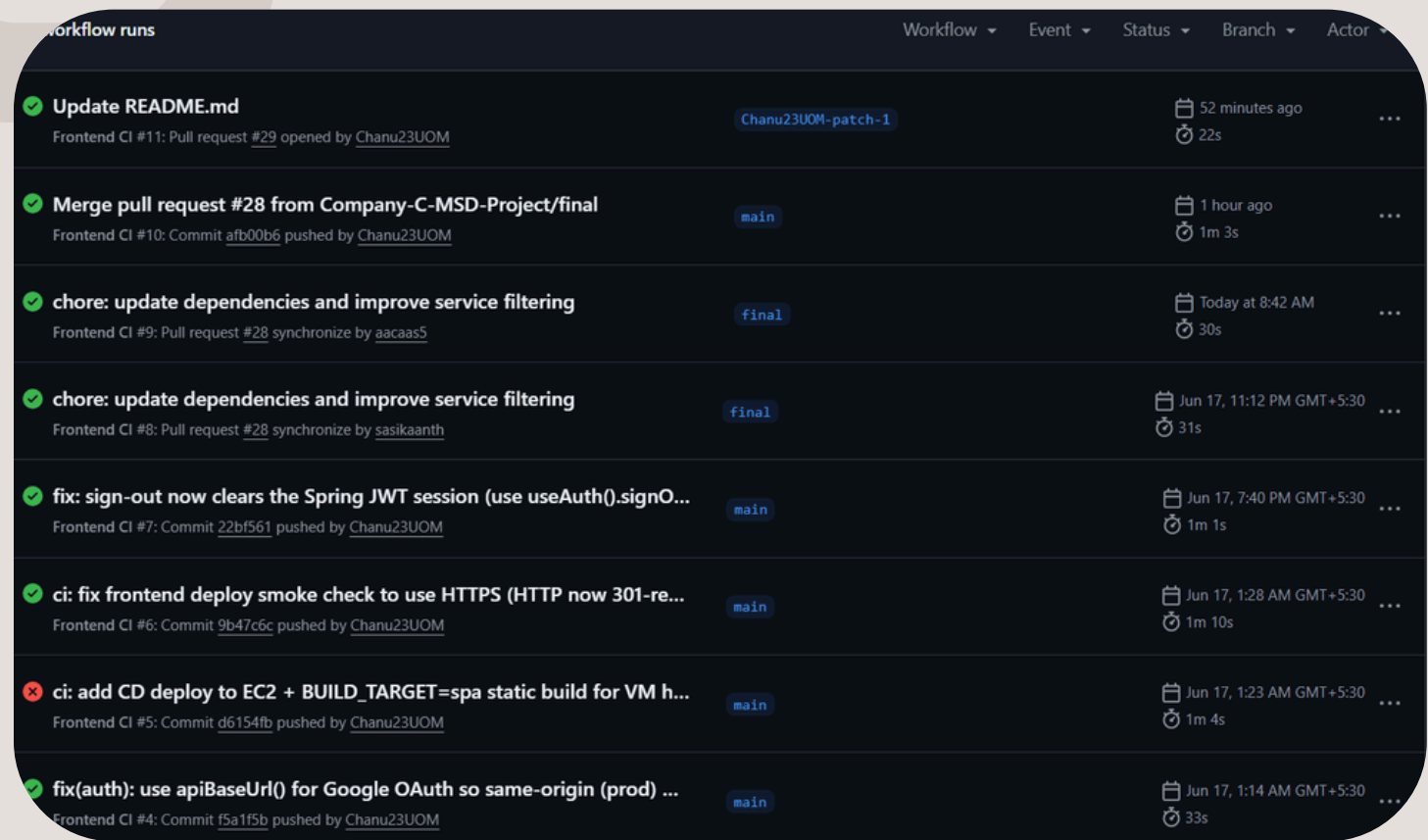
Code Quality

Lombok @RequiredArgsConstructor, @Getter/@Setter reduce boilerplate. @Valid bean validation on all request DTOs. Centralised error handling.

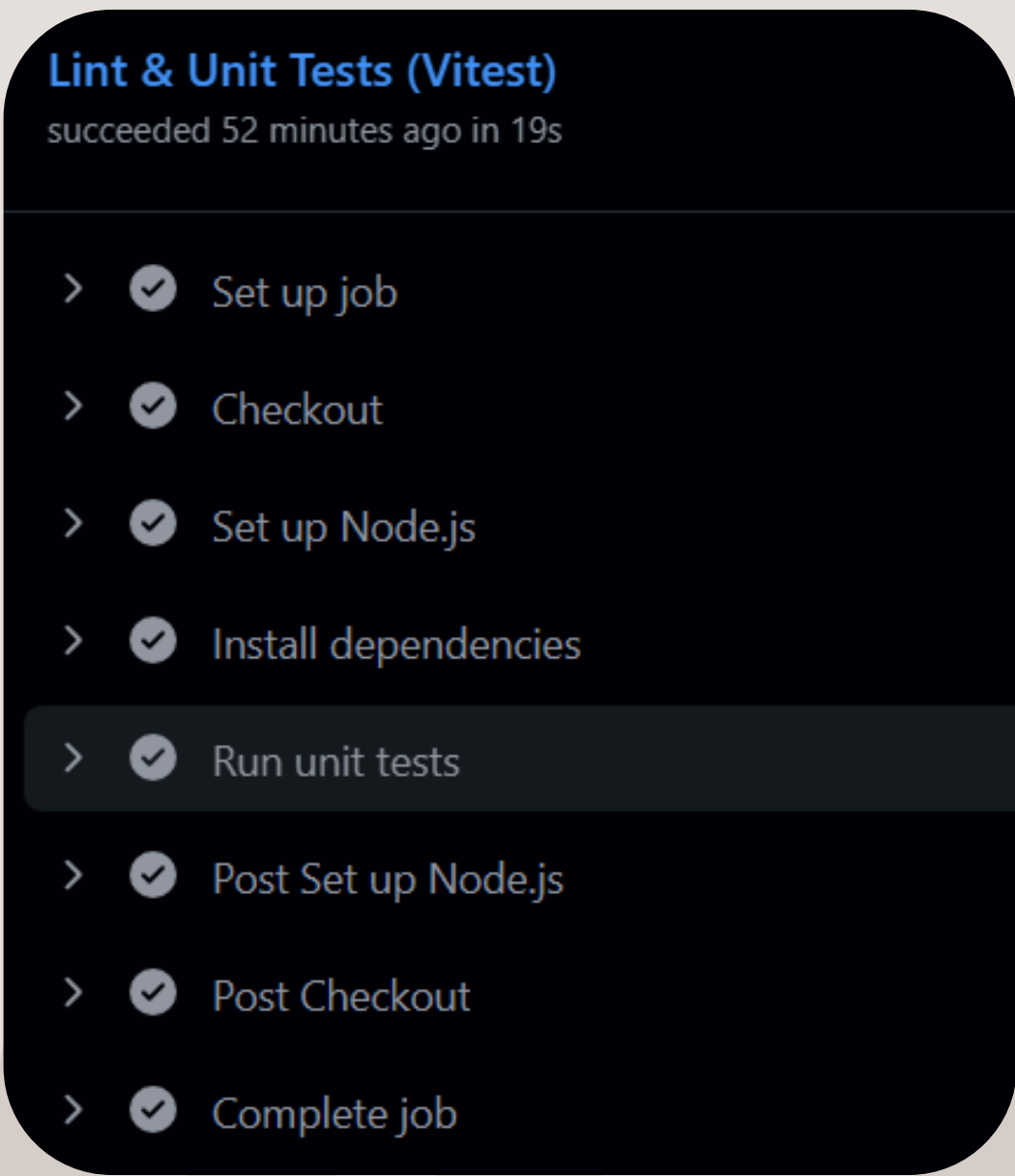
Project Management & Development Approach



Use of phpMyAdmin for development and testing of the DB



Use of GitHub workflow management, gh cli and Live smoke and unit test automation



Highlights

19

**Contributors for
Backend**

22

**Contributors for
Frontend**

23

**Total merged pull
requests**

Contribution of all 39 members of the Company

A Concrete End-to-End Journey - Putting It All Together

1

Visitor opens <https://fixitnow.chanupahansaja.me>

nginx serves React SPA over HTTPS (Let's Encrypt TLS)

2

Signs up as homeowner

POST /auth/signup → BCrypt hash → User saved → access + refresh tokens issued

3

Browses Plumbing category

GET /service-categories + SmartMatchService → providers ranked by rating + Haversine proximity

4

Creates a booking

POST /bookings with JWT → JwtAuthFilter → BookingV1Service → PENDING booking + notification

5

Provider accepts → starts → completes

PATCH .../status (3 transitions) → each validated by state machine → notifications at each step

6

Stripe payment processed

PaymentIntent created → webhook confirms → wallet updated → both parties notified

7

Homeowner leaves review

POST /reviews → provider avg rating + badge recalculated → visible on provider profile

8

Every write = a row change in MySQL

Every protected call authorised by role. Every error → clean JSON via GlobalExceptionHandler.

System Summary

16

Database Tables

14

JPA Repository
Interfaces

3

User Roles (RBAC)

5

Booking State
Transitions

4

Test Layers (Unit→E2E)

7

CI/CD Pipeline Steps

8

OOP Concepts
Demonstrated

8

Core Service Categories

5

IT Integration Test
Workflows

100%

HTTPS in Production

Gantt Chart

FixItNow | On-Demand Home Repair & Maintenance Platform / Team C

Project Gantt Chart | Mar 24 – Jun 16, 2026 | 12 Weeks

LEGEND — Phase Color: Requirements Design Development Integration Testing Deployment Documentation Closure

[illegible]

Gantt Chart

[illegible]

Thank You

FixItNow : On-Demand Home Repair & Maintenance Platform



HOME SERVICES, SIMPLIFIED

**Reliable repairs.
Right at your doorstep.**

FixItNow connects homeowners with local professionals — electricians, plumbers, masons, carpenters and more. Book your service on completion. Backed by a 100% satisfaction guarantee.